FRENCH UNIVERSITY IN ARMENIA

Faculty of Computer Science and Applied Mathematics

Subject Databases 1 | Database Modeling and Implementation

Group Hambik HAYRAPETIAN

Narek MADATYAN

Andranik KHACHATRYAN

Professor Varazdat AVETISYAN, PhD

06.04.2026

Contents

1. **Conceptual Design** 3
   - 1.1 **Analyze the problem domain** 3
   - 1.2 **Identify main data objects and business rules** 3
   - 1.3 **Define system data requirements and processes** 5
     - 1.3.1 **Draw flow-charts (Use-case, Activity & Sequence Diagrams)** 8
2. **Logical Design** 11
   - 2.1 **Develop an Entity–Relationship (ER) diagram** 11
   - 2.2 **Identify entities, attributes, and relationships** 11
   - 2.3 **Define relationship cardinalities and constraints** 14
   - 2.4 **Transform the ER model into a relational schema** 16
   - 2.5 **Define tables, primary keys, and foreign keys** 16
   - 2.6 **Apply the normalization process (1NF, 2NF, 3NF)** 19
   - 2.7 **Apply relational algebra operations (selection, projection, join, union, intersection, difference): Query Text + relational algebra operation** 20
3. **Physical Implementation** 24
   - 3.1 **Implement the database using SQL** 24
   - 3.2 **Create database objects using DDL statements** 24
   - 3.3 **Insert and manage data using DML statements (for each table more than 40 records)** 27
   - 3.4 **Retrieve information using DQL queries & Implement Indexes to optimize query performance** 40
   - 3.5 **Apply SQL queries and store results in Views** 48
   - 3.6 **Implement Triggers to automate database operations and enforce business rules** 50
   - 3.7 **Develop Stored Procedures for reusable database logic and operations** 53
   - 3.8 **Implement access control using DCL statements** 56
   - 3.9 **Connect the database to an application or interface** 58
4. **Conclusion** 64

1. Conceptual Design

1.1 Analyze the problem domain

The system is an online platform where users can create accounts, buy digital games, and store them in their personal library. Each purchase is recorded in the database and linked to the user's profile. The platform also supports social features: users can add friends, and they can write reviews and give ratings for games. All these interactions are managed through the database, which keeps track of users, games, friendships, and reviews to ensure smooth operation and reliable data management.

1.2 Identify main data objects and business rules

The system is designed as an online platform where users can purchase games, manage their personal library, and interact socially through friendships and reviews. To support these functions, the database must capture key data objects and enforce business rules that ensure consistency and reliability.

Main Data Objects

- **User –** stores account details and profile information of platform users.
- **Publisher –** represents companies or individuals responsible for publishing and distributing games.
- **Developer –** represents individuals or teams that design and develop games.
- **Game** – stores information about games, including name, version, price, storage size (e.g., MB/GB), and other relevant attributes.
- **Genre –** represents categories or classifications of games (e.g., Action, RPG, Strategy).

- **Order –** records user purchases and contains order-related details such as date and total amount.
- **Payment –** stores transaction details associated with orders.
- **Review –** represents user feedback on games, including ratings and comments.
- **Library –** represents games owned by a user, including attributes such as playtime and favorite status.
- **Wishlist –** stores games that a user intends to purchase in the future.

Business Rules

1. A user must register and create an account before accessing the system.
2. Each order is associated with exactly one user, and a user can place multiple orders.
3. Each order must contain one or more games, and a game can appear in 0 or more orders.
4. Each order generates exactly one payment transaction that records the payment details.
5. A user cannot own duplicate copies of the same game in their library.
6. A game can be owned by multiple users.
7. A user can review a game only if they own that game.
8. Each game must be associated with exactly one publisher, while a publisher can publish multiple games.
9. Each game must be associated with one or more developers, and a developer can work on multiple games.

1.3 Define system data requirements and processes

Data Requirements

The database must store and manage the following information:

- **Users:**
  - nickname - display name shown to other users on the platform.
  - email - stores the user's email address; optional since not all users may provide it.
  - balance - represents the user's available funds for purchases.
- **Order:**
  - orderDate - date when the order was created.
  - quantity – number of games that one order contains.

- **Payment:**
  - amount - total amount paid.
  - paymentMethod – method used (e.g., card, wallet).
- **Publisher:**
  - publisherName - name of the publishing company or individual.
  - website – official website; optional because not all publishers may have one.
- **Developer:**
  - developerName - name of the developer or development team.
  - email – contact email; optional since it may not always be available.
- **Library:**
  - favorite - indicates whether the game is marked as a favorite.
  - playtime – total time the user has spent playing the game.
- **Game:**
  - version - indicates the current version of the game.
  - gamePrice – cost of the game.
  - gameSize – storage size required for installation (e.g., GB).
  - downloadURL - link used to download the game.
  - gameName - name/title of the game.
- **Genre:**
  - genreName - name of the genre (e.g., Action, RPG).
- **Review:**
  - reviewDate - date the review was posted.
  - description - textual feedback provided by the user.

- rating - numerical evaluation of the game.

Processes

1. Users create accounts and authenticate to access the system.
2. Users browse available games and place orders to purchase them.
3. Each order is recorded in the system and linked to the corresponding user.
4. Orders generate a payment transaction, ensuring that payment details are stored and tracked.
5. Purchased games are added to the user's library.
6. Users can add games to their wishlist for future purchases.
7. Users can write reviews and assign ratings to games they own.
8. Games are associated with one publisher and one or more developers.

1.3.1 Draw flow-charts (Use-case, Activity & Sequence Diagrams)

Figure 1: Use-case

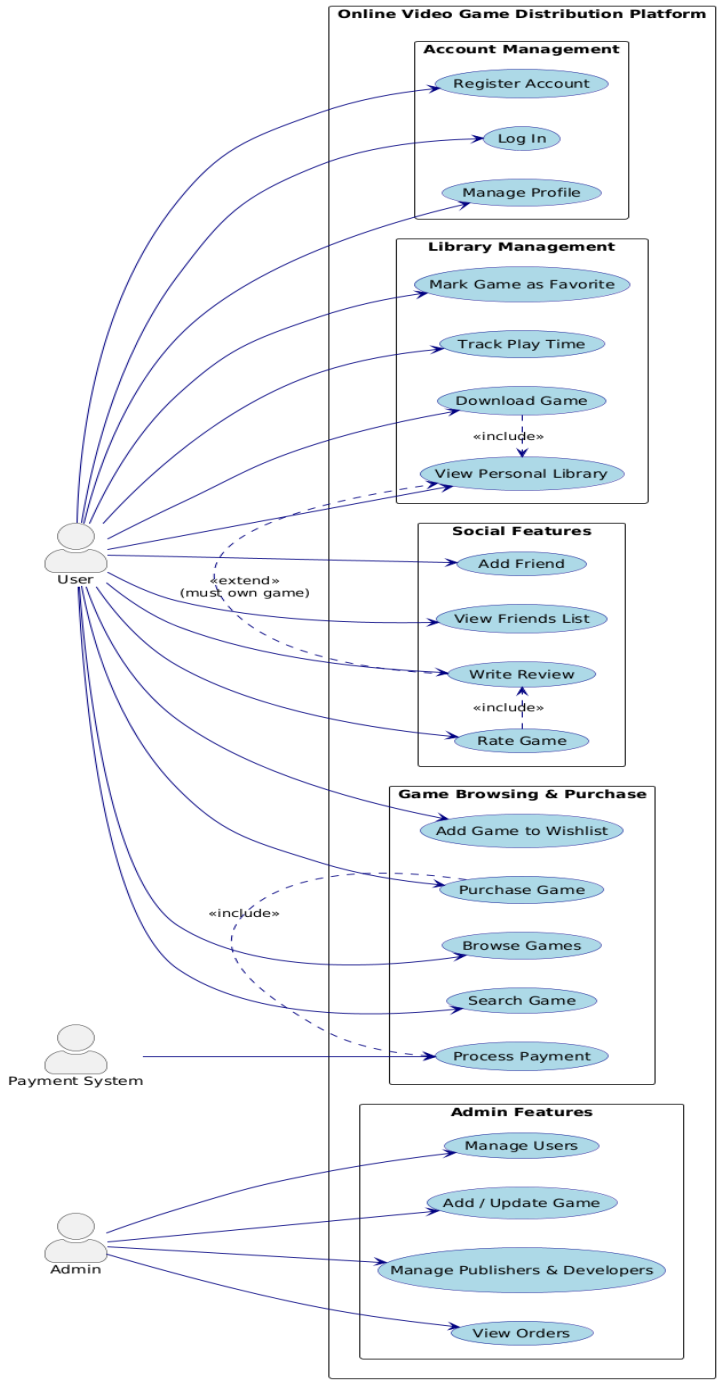


Figure 2: Activity Diagram

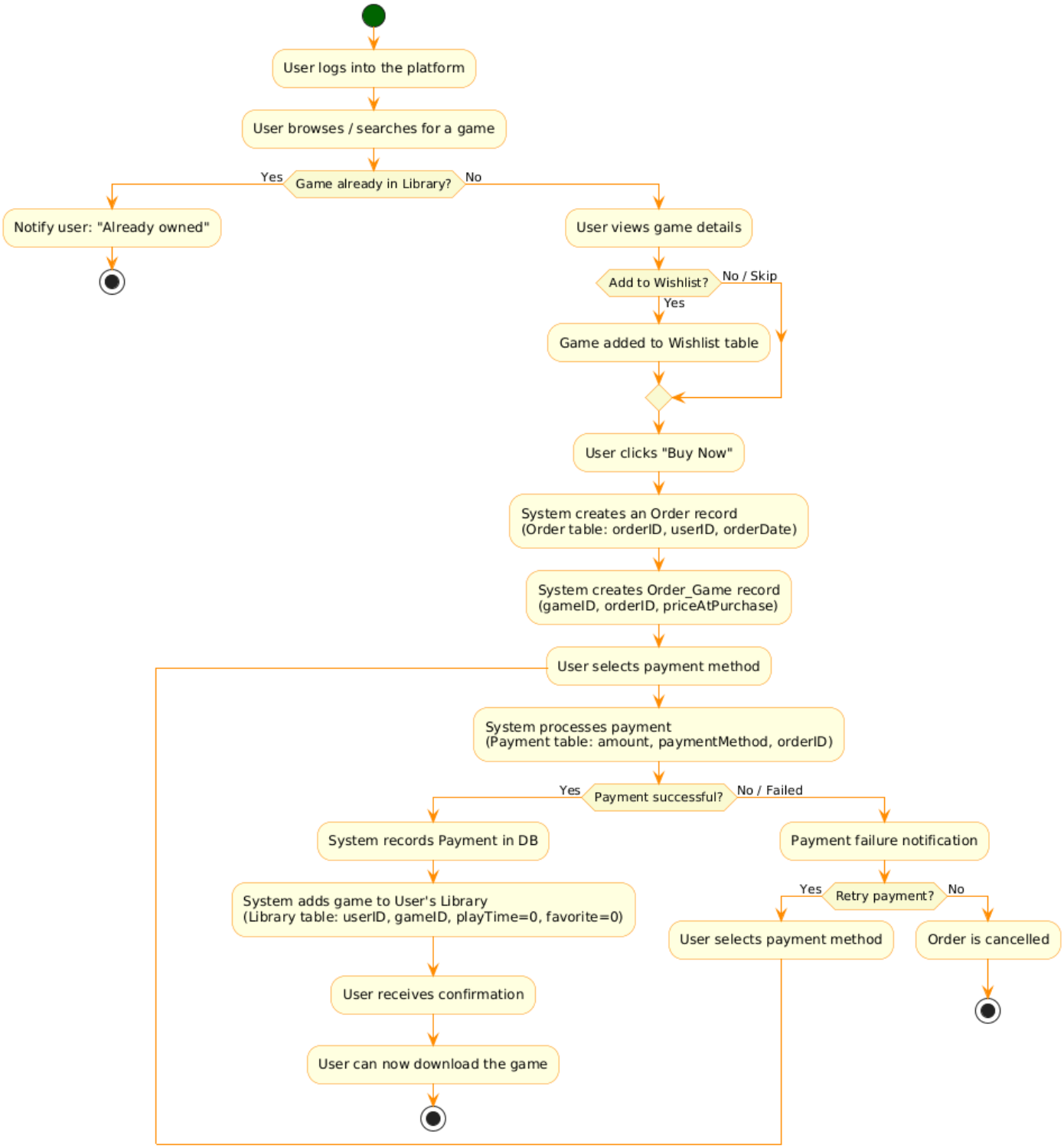
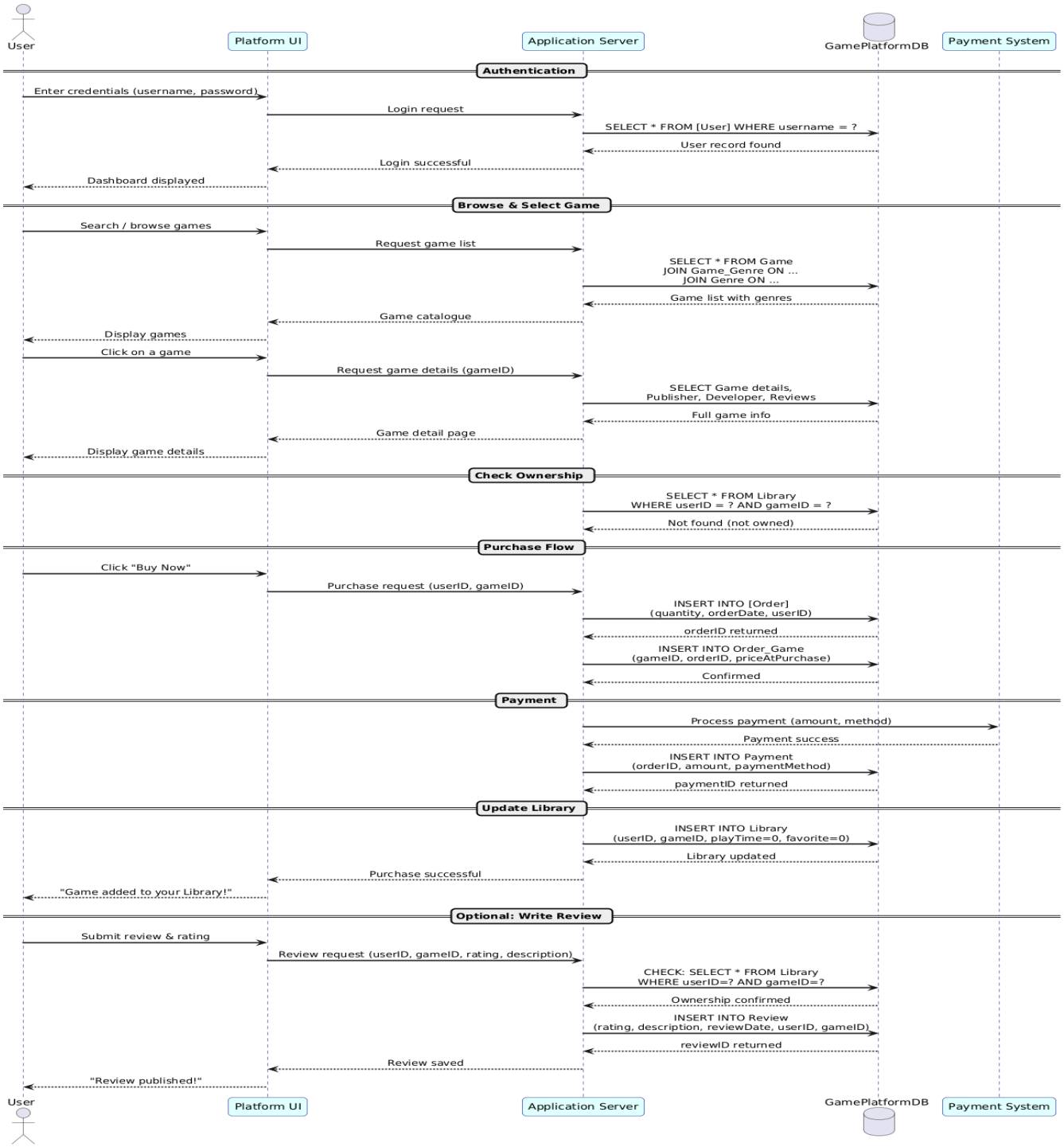
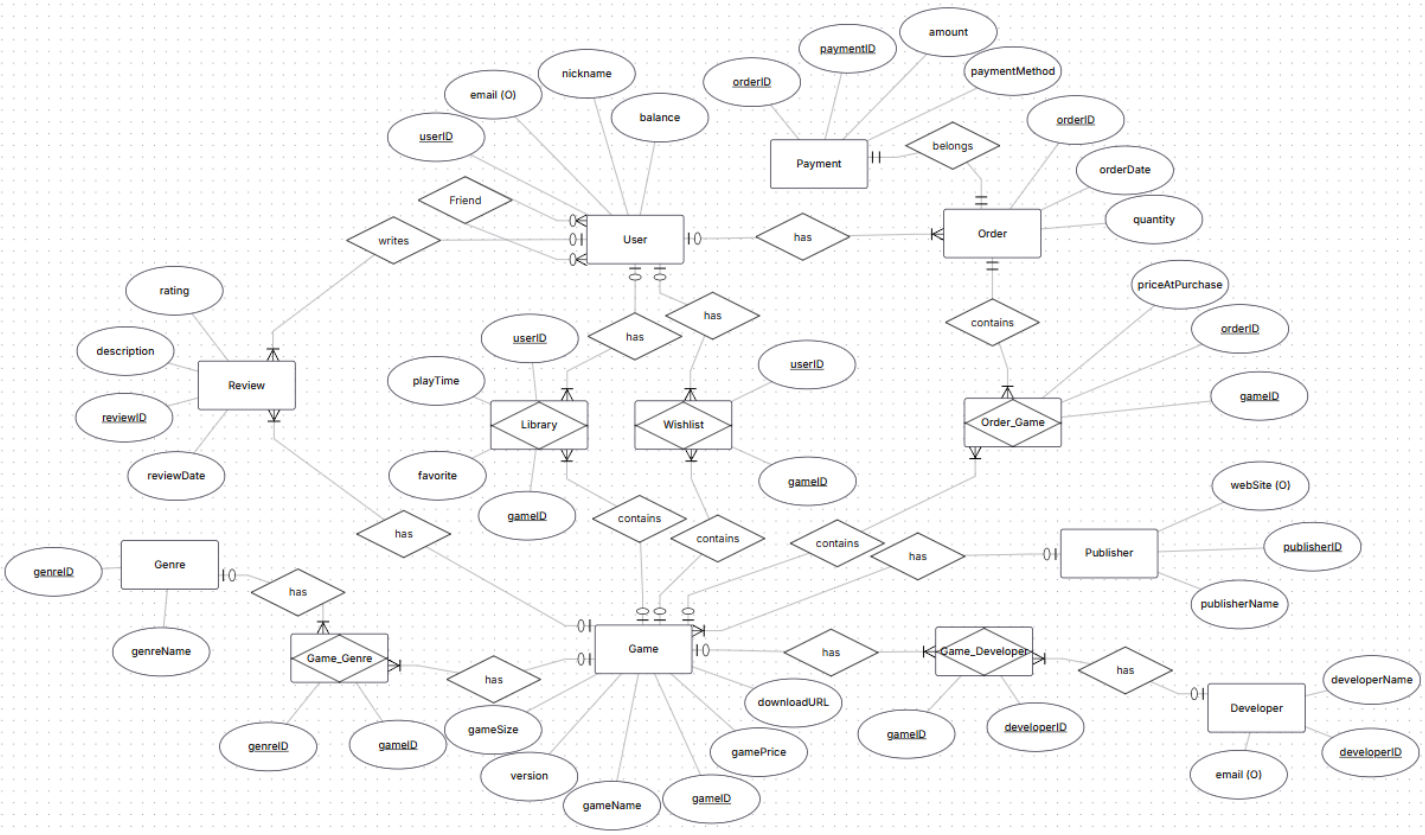


Figure 1: Sequence Diagram



2. Logical Design

2.1 Develop an Entity–Relationship (ER) diagram



2.2 Identify entities, attributes, and relationships

1. User: Represents a registered user of the platform
    - userID (Primary Key),
    - nickname,
    - Email (Optional),
    - Balance
2. Game: Represents a digital game available for purchase or in users' libraries
    - gameID (Primary Key),
    - gameSize,

- version,
- gameName,
- gamePrice,
- downloadURL,
- publisherID (Foreign Key)

3. Developer: Represents the developers responsible for creating games
    - developerID (Primary Key),
    - developerName,
    - email (Optional)
4. Publisher: Represents the companies that publish the games
    - publisherID (Primary Key),
    - publisherName,
    - website (Optional)
5. Genre: Represents a category or genre of games
    - genreID (Primary Key),
    - genreName
6. Library (Associative Entity): Represents the list of the games that the user owns
    - userID (Foreign Key, part of composite Primary Key),
    - gameID (Foreign Key, part of composite Primary Key),
    - playtime,
    - favorite
7. Wishlist (Associative Entity): Represents games that a user has added to their wishlist
    - userID (Foreign Key, part of composite Primary Key),

- gameID (Foreign Key, part of composite Primary Key)

8. Review: Represents user-written reviews and ratings for games
    - reviewID (Primary Key),
    - rating,
    - description,
    - reviewDate,
    - userID (Foreign Key),
    - gameID (Foreign Key)
9. Order: Represents a user's purchase transaction
    - orderID (Primary Key),
    - orderDate,
    - totalPrice (removed during normalization — see Section 2.6),
    - userID (Foreign Key)
10. Order_Game (Associative Entity): Represents the games included in a specific order and their purchase details
    - orderID (Foreign Key, part of composite Primary Key),
    - gameID (Foreign Key, part of composite Primary Key),
    - priceAtPurchase
11. Game_Genre (Associative Entity): Associates games with one or more genres
    - gameID (Foreign Key, part of composite Primary Key),
    - genreID (Foreign Key, part of composite Primary Key)
12. Game_Developer (Associative Entity): Associates games with one or more developers
    - developerID (Foreign Key, part of composite Primary Key),

- gameID (Foreign Key, part of composite Primary Key)

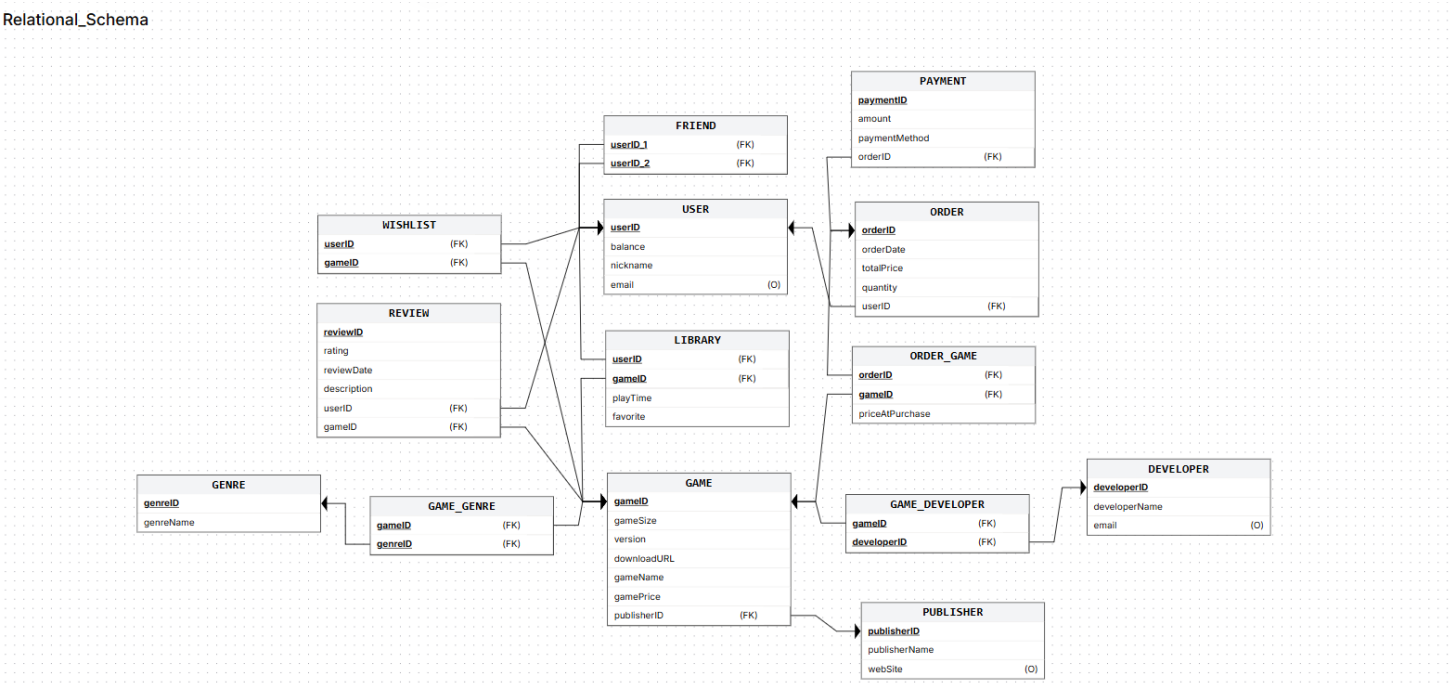
2.3 Define relationship cardinalities and constraints

- User ↔ Library
  - One user may have zero or more library entries (optional for user),
  - Each library entry must belong to exactly one user (mandatory for library),
  - Many-to-many relationship realized through Library associative entity
- Game ↔ Library
  - One game may appear in zero or more users' libraries (optional for games),
  - Each library entry must refer to exactly one game (mandatory for library)
- User ↔ Wishlist
  - One user may have zero or more wishlist entries (optional for user),
  - Each wishlist entry must belong to exactly one user (mandatory for wishlist)
- Game ↔ Wishlist
  - One game may appear in zero or more wishlists (optional for game),
  - Each wishlist entry must refer to exactly one game (mandatory for wishlist)
- User ↔ Review
  - One user may write zero or more reviews (optional for user),
  - Each review must be written by exactly one user (mandatory for review)
- Game ↔ Review
  - One game may have zero or more reviews (optional for game),
  - Each review must refer to exactly one game (mandatory for review)
- Game ↔ Game_Genre ↔ Genre
  - One game must have at least one genre (mandatory for game),

- One genre may be associated with zero or more games (optional for genre)

- Game ↔ Game_Developer ↔ Developer
  - One game must have at least one developer (mandatory for game),
  - One developer may have zero or more games (optional for developer)
- Publisher ↔ Game
  - One publisher may have zero or more games (optional for publisher),
  - Each game must have exactly one publisher (mandatory for game)
- User ↔ Order
  - One user may have zero or more orders (optional for user),
  - Each order must belong to exactly one user (mandatory for order)
- Order ↔ Order_Game ↔ Game
  - One order must contain one or more games (mandatory for order),
  - Each Order_Game entry must refer to exactly one order and one game (mandatory for order_game)

2.4 Transform the ER model into a relational schema



2.5 Define tables, primary keys, and foreign keys

Table	Column	Data Type	Key	Description
User	**`userID`**	`VARCHAR(50)`	**PK**	Unique username — primary identifier
	`nickname`	`VARCHAR(50)`		Display name
	`email`	`VARCHAR(100)`		Contact email address
	`balance`	`DECIMAL(10,2)`		User's balance
Publisher	**`publisherID`**	`INT IDENTITY`	**PK**	Auto-incremented publisher identifier
	`publisherName`	`VARCHAR(100)`		Name of the publishing company
	`webSite`	`VARCHAR(200)`		Publisher website URL
Developer	**`developerID`**	`INT IDENTITY`	**PK**	Auto-incremented developer identifier
	`developerName`	`VARCHAR(100)`		Studio name
	`email`	`VARCHAR(100)`		Developer contact email

Table	Column	Data Type	Key	Description
Game	**gameID**	INT IDENTITY	**PK**	Auto-incremented game identifier
	gameName	VARCHAR(100)		Title of the game
	gamePrice	DECIMAL(10,2)		Retail price
	gameSize	INT		Size in MB
	version	VARCHAR(20)		Current version string
	downloadURL	VARCHAR(200)		Download link
	publisherID	INT	**FK**	References Publisher(publisherID)
Genre	**genreID**	INT IDENTITY	**PK**	Auto-incremented genre identifier
	genreName	VARCHAR(50)		Genre name (e.g. RPG, FPS)
Game_Genre	**gameID**	INT	**PK, FK**	References Game(gameID)
	genreID	INT	**PK, FK**	References Genre(genreID)
Game_Developer	**developerID**	INT	**PK, FK**	References Developer(developerID)
	gameID	INT	**PK, FK**	References Game(gameID)
Friend	**userID_1**	VARCHAR(50)	**PK, FK**	References User(username) — first user
	userID_2	VARCHAR(50)	**PK, FK**	References User(username) — second user
Library	**userID**	VARCHAR(50)	**PK, FK**	References User(username)
	gameID	INT	**PK, FK**	References Game(gameID)
	playTime	INT		Total minutes played

Table	Column	Data Type	Key	Description
	favorite	BIT		User marked as favorite
Wishlist	**userID**	VARCHAR(50)	**PK, FK**	References User(username)
	gameID	INT	**PK, FK**	References Game(gameID)
Review	**reviewID**	INT IDENTITY	**PK**	Auto-incremented review identifier
	userID	VARCHAR(50)	**FK**	References User(username)
	gameID	INT	**FK**	References Game(gameID)
	rating	INT		Score 1–10
	description	TEXT		Written review content
	reviewDate	DATE		Date submitted
Order	**orderID**	INT IDENTITY	**PK**	Auto-incremented order identifier
	userID	VARCHAR(50)	**FK**	References User(username)
	quantity	INT		Number of games in order
	orderDate	DATE		Date placed
Order_Game	**gameID**	INT	**PK, FK**	References Game(gameID)
	orderID	INT	**PK, FK**	References Order(orderID)
	priceAtPurchase	DECIMAL(10,2)		Price locked at purchase time
Payment	**paymentID**	INT IDENTITY	**PK**	Auto-incremented payment identifier
	orderID	INT	**FK**	References Order(orderID)
	amount	DECIMAL(10,2)		Payment amount

Table	Column	Data Type	Key	Description
	paymentMethod	VARCHAR(50)		Method: Credit Card, PayPal, Debit, Free

2.6 Apply the normalization process (1NF, 2NF, 3NF)

- *First Normal Form (1NF)*
  1. Checked all tables to make sure each column contains indivisible values. For example, gameName, downloadURL, gamePrice in Game are atomic.
  2. Separated multivalued attributes. For example, genres in Game were moved to a separate associative entity Game_Genre to avoid storing multiple genres in a single column.
  3. All associative entities (Library, Wishlist, Order_Game, Game_Developer, Game_Genre) already store one relationship per row, ensuring no repeating groups.
- *Second Normal Form (2NF)*
  1. Identified tables with composite keys:
     - Library (userID, gameID)
     - Wishlist (userID, gameID)
     - Order_Game (orderID, gameID)
     - Game_Developer (developerID, gameID)
     - Game_Genre (gameID, genreID)
  2. Verified that all non-key attributes depend on the full composite key, not just part of it. For example, playTime in Library depends on both userID and gameID.
  3. Single-key tables like User, Game, Developer, Publisher, Review, and Order automatically satisfy 2NF.

- *Third Normal Form (3NF)*

  1. Checked each table for attributes that depend on other non-key attributes. For example, in Game, all attributes (gameName, gameSize, version, downloadURL, gamePrice) depend only on gameID. The publisherID is a foreign key, not a dependent attribute, so no transitive dependency exists.
  2. In Order, removed the totalPrice attribute, which would have been derived from priceAtPurchase in Order_Game. Now, the Payment entity stores the actual amount for each order, eliminating a transitive dependency between Order and User.
  3. In Order_Game, priceAtPurchase depends on both orderID and gameID, correctly handling any many-to-many relationship without introducing redundancy.

2.7 Apply relational algebra operations (selection, projection, join, union, intersection, difference): Query Text + relational algebra operation

#	Query Title	Relational Algebra
1	Select free games	$\sigma(gamePrice=0)(Game)$
2	Select games priced above $50	$\sigma(gamePrice>50)(Game)$
3	Select users from a specific email domain	$\sigma(email\ LIKE\ '\%gmail.com')([User])$
4	Select reviews with rating = 10	$\sigma(rating=10)(Review)$
5	Select orders placed in	$\sigma(orderDate\geq'2024-01-01' \wedge orderDate\leq'2024-01-31')([Order])$

#	Query Title	Relational Algebra
	January 2024	
6	Project game names and prices	π(gameName, gamePrice)(Game)
7	Project usernames and emails	π(username, email)([User])
8	Project publisher names and websites	π(publisherName, webSite)(Publisher)
9	Project developer names	π(developerName)(Developer)
10	Project genre names	π(genreName)(Genre)
11	Join Game with Publisher	π(gameName,publisherName)(Game ⋈publisherID Publisher)
12	Join User with Library and Game	π(username,gameName)([User] ⋈username=userID Library ⋈gameID Game)
13	Join Game with Genre via Game_Genre	π(gameName,genreName)(Game ⋈ Game_Genre ⋈ Genre)
14	Join Order with Payment	π(orderID,amount,paymentMethod)([Order] ⋈orderID Payment)
15	Join User with Review and Game	π(username,gameName,rating)([User] ⋈ Review ⋈ Game)
16	Union: users who own OR wishlist a game (gameID=1)	σ(gameID=1)π(userID)(Library) ∪ σ(gameID=1)π(userID)(Wishlist)

#	Query Title	Relational Algebra
17	Union: all userIDs appearing in Library or Order	π(userID)(Library) ∪ π(userID)([Order])
18	Intersection: users who own AND have wishlisted any game	π(userID)(Library) ∩ π(userID)(Wishlist)
19	Intersection: users who placed an order AND wrote a review	π(userID)([Order]) ∩ π(userID)(Review)
20	Difference: users who have a library entry but no wishlist entry	π(userID)(Library) − π(userID)(Wishlist)
21	Difference: users who placed no orders	π(username)([User]) − π(userID)([Order])
22	Selection + Projection: names of RPG games	π(gameName)(σ(genreName='RPG')(Game ⋈ Game_Genre ⋈ Genre))
23	Join + Selection: users who own Cyberpunk 2077	π(username)(σ(gameName='Cyberpunk 2077')([User] ⋈ Library ⋈ Game))
24	Aggregation: average rating per game	γ(gameID; AVG(rating))(Review)

#	Query Title	Relational Algebra
25	Top 5 most expensive games	$\pi(gameName,gamePrice)(\sigma TOP5(\sigma(sorted)(\pi(gameName,gamePrice)(Game))))$
26	Total spent per user	$\gamma(userID;\ SUM(priceAtPurchase))([Order] \bowtie Order_Game)$

3. Physical Implementation

3.1 Implement the database using SQL

```
-- Create the database
CREATE DATABASE GamePlatformDB;
GO

-- Switch to the new database
USE GamePlatformDB;
GO
```

3.2 Create database objects using DDL statements

```
CREATE TABLE [User] (
    username VARCHAR(50) PRIMARY KEY,  -- No need for IDENTITY for VARCHAR
primary keys
    nickname VARCHAR(50),
    email VARCHAR(100),
    balance DECIMAL(10,2)
);

CREATE TABLE Publisher (
    publisherID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for
publisherID
    publisherName VARCHAR(100),
    webSite VARCHAR(200)
);

CREATE TABLE Developer (
    developerID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for
developerID
    developerName VARCHAR(100),
    email VARCHAR(100)
);

CREATE TABLE Game (
    gameID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for gameID
    gameSize INT,
    version VARCHAR(20),
    downloadURL VARCHAR(200),
```

```
    gameName VARCHAR(100),
    gamePrice DECIMAL(10,2),
    publisherID INT,
    FOREIGN KEY (publisherID) REFERENCES Publisher(publisherID)
);

CREATE TABLE Genre (
    genreID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for genreID
    genreName VARCHAR(50)
);

CREATE TABLE Game_Genre (
    gameID INT,
    genreID INT,
    PRIMARY KEY (gameID, genreID),
    FOREIGN KEY (gameID) REFERENCES Game(gameID),
    FOREIGN KEY (genreID) REFERENCES Genre(genreID)
);

CREATE TABLE Game_Developer (
    developerID INT,
    gameID INT,
    PRIMARY KEY (developerID, gameID),
    FOREIGN KEY (developerID) REFERENCES Developer(developerID),
    FOREIGN KEY (gameID) REFERENCES Game(gameID)
);

CREATE TABLE Friend (
    userID_1 VARCHAR(50),
    userID_2 VARCHAR(50),
    PRIMARY KEY (userID_1, userID_2),
    FOREIGN KEY (userID_1) REFERENCES [User](username),
    FOREIGN KEY (userID_2) REFERENCES [User](username)
);

CREATE TABLE Library (
    userID VARCHAR(50),
    gameID INT,
    playTime INT,
    favorite BIT,
    PRIMARY KEY (userID, gameID),
    FOREIGN KEY (userID) REFERENCES [User](username),
    FOREIGN KEY (gameID) REFERENCES Game(gameID)
);
```

```sql
CREATE TABLE Wishlist (
    userID VARCHAR(50),
    gameID INT,
    PRIMARY KEY (userID, gameID),
    FOREIGN KEY (userID) REFERENCES [User](username),
    FOREIGN KEY (gameID) REFERENCES Game(gameID)
);

CREATE TABLE Review (
    reviewID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for reviewID
    rating INT,
    description TEXT,
    reviewDate DATE,
    userID VARCHAR(50),
    gameID INT,
    FOREIGN KEY (userID) REFERENCES [User](username),
    FOREIGN KEY (gameID) REFERENCES Game(gameID)
);

CREATE TABLE [Order] (
    orderID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for orderID
    quantity INT,
    orderDate DATE,
    userID VARCHAR(50),
    FOREIGN KEY (userID) REFERENCES [User](username)
);

CREATE TABLE Order_Game (
    gameID INT,
    orderID INT,
    priceAtPurchase DECIMAL(10,2),
    PRIMARY KEY (gameID, orderID),
    FOREIGN KEY (gameID) REFERENCES Game(gameID),
    FOREIGN KEY (orderID) REFERENCES [Order](orderID)
);

CREATE TABLE Payment (
    paymentID INT IDENTITY(1,1) PRIMARY KEY,  -- Set IDENTITY for paymentID
    orderID INT,
    amount DECIMAL(10,2),
    paymentMethod VARCHAR(50),
    FOREIGN KEY (orderID) REFERENCES [Order](orderID)
);
```

The following DDL statements are used to define the structure of the GamePlatformDB database. They create all required tables, attributes, and constraints, ensuring data integrity and supporting relationships between entities.

3.3 Insert and manage data using DML statements (for each table more than 40 records)

```
USE GamePlatformDB;

INSERT INTO [User] VALUES
('alex_k','Alex','alexk@gmail.com',120.50),
('maria_dev','Maria','maria.dev@gmail.com',250.00),
('john_gamer','John','john.gamer@gmail.com',75.25),
('luna_play','Luna','luna.play@gmail.com',180.00),
('pixelking','PixelKing','pixelking@gmail.com',300.99),
('shadowfox','ShadowFox','shadowfox@gmail.com',95.40),
('nova_star','Nova','nova.star@gmail.com',210.75),
('armin_kh','Armin','armin.kh@gmail.com',50.00),
('tigergamer','Tiger','tiger.gamer@gmail.com',400.00),
('nightcoder','NightCoder','nightcoder@gmail.com',60.60),
('stormblade','StormBlade','stormblade@gmail.com',130.30),
('silentwolf','SilentWolf','wolf@gmail.com',85.00),
('cyberhero','CyberHero','cyberhero@gmail.com',500.00),
('dragonlord','DragonLord','dragon@gmail.com',275.45),
('retroplayer','RetroPlayer','retro@gmail.com',40.00),
('skygamer','SkyGamer','sky@gmail.com',150.00),
('ironmaniac','IronManiac','iron@gmail.com',320.80),
('darkrider','DarkRider','dark@gmail.com',110.10),
('quantumplay','QuantumPlay','quantum@gmail.com',220.00),
('vortexgamer','Vortex','vortex@gmail.com',99.99),
('phantombyte','PhantomByte','phantom@gmail.com',135.75),
('lunarfox','LunarFox','lunar@gmail.com',88.88),
('silverarrow','SilverArrow','silver@gmail.com',145.60),
('techwizard','TechWizard','tech@gmail.com',260.00),
('doomplayer','DoomPlayer','doom@gmail.com',70.00),
```

```
('stealthninja','StealthNinja','ninja@gmail.com',180.80),
('orbitgamer','Orbit','orbit@gmail.com',95.95),
('hypernova','HyperNova','hyper@gmail.com',310.10),
('bytehunter','ByteHunter','hunter@gmail.com',55.55),
('skyfall','SkyFall','skyfall@gmail.com',140.00),
('stormrider','StormRider','stormrider@gmail.com',175.25),
('pixelstorm','PixelStorm','pixelstorm@gmail.com',200.00),
('auroragamer','Aurora','aurora@gmail.com',125.75),
('voidwalker','VoidWalker','void@gmail.com',90.00),
('frostgamer','Frost','frost@gmail.com',60.00),
('cyberninja','CyberNinja','cyberninja@gmail.com',210.00),
('neonplayer','Neon','neon@gmail.com',80.00),
('ultragamer','Ultra','ultra@gmail.com',330.30),
('gamemaster','GameMaster','gm@gmail.com',500.50),
('arcadehero','ArcadeHero','arcade@gmail.com',140.40),
('binaryknight','BinaryKnight','binary@gmail.com',190.90),
('gamelord','GameLord','lord@gmail.com',275.00),
('alphaomega','AlphaOmega','alpha@gmail.com',160.60),
('cosmicplayer','Cosmic','cosmic@gmail.com',220.20),
('vectorx','VectorX','vector@gmail.com',145.00),
('speedrunner','SpeedRunner','speed@gmail.com',75.00),
('battleking','BattleKing','battle@gmail.com',260.75),
('arcadefox','ArcadeFox','arcfox@gmail.com',110.00),
('nebulagamer','Nebula','nebula@gmail.com',205.00),
('legendplayer','Legend','legend@gmail.com',350.00);
INSERT INTO Publisher VALUES
('Ubisoft','https://ubisoft.com'),
('Electronic Arts','https://ea.com'),
('Valve','https://valvesoftware.com'),
('Bethesda Softworks','https://bethesda.net'),
('CD Projekt','https://cdprojekt.com'),
('Rockstar Games','https://rockstargames.com'),
('Square Enix','https://square-enix.com'),
('Bandai Namco','https://bandainamcoent.com'),
('Nintendo','https://nintendo.com'),
('Sony Interactive Entertainment','https://playstation.com'),
('Sega','https://sega.com'),
('Capcom','https://capcom.com'),
```

```
('Konami','https://konami.com'),
('Paradox Interactive','https://paradoxinteractive.com'),
('THQ Nordic','https://thqnordic.com'),
('2K Games','https://2k.com'),
('Warner Bros Games','https://wbgames.com'),
('Focus Entertainment','https://focus-entmt.com'),
('Deep Silver','https://deep-silver.com'),
('505 Games','https://505games.com'),
('Gearbox Publishing','https://gearboxpublishing.com'),
('Team17','https://team17.com'),
('Annapurna Interactive','https://annapurnainteractive.com'),
('Raw Fury','https://rawfury.com'),
('Devolver Digital','https://devolverdigital.com'),
('TinyBuild','https://tinybuild.com'),
('Coffee Stain Publishing','https://coffeestainstudios.com'),
('Frontier Developments','https://frontier.co.uk'),
('Bohemia Interactive','https://bohemia.net'),
('Krafton','https://krafton.com'),
('Pearl Abyss','https://pearlabyss.com'),
('Smilegate','https://smilegate.com'),
('NetEase Games','https://neteasegames.com'),
('Tencent Games','https://tencentgames.com'),
('MiHoYo','https://mihoyo.com'),
('Grinding Gear Games','https://grindinggear.com'),
('Hello Games','https://hellogames.org'),
('Re-Logic','https://re-logic.com'),
('Supergiant Games','https://supergiantgames.com'),
('Larian Studios','https://larian.com'),
('Insomniac Games','https://insomniac.games'),
('Naughty Dog','https://naughtydog.com'),
('FromSoftware','https://fromsoftware.jp'),
('id Software','https://idsoftware.com'),
('Arkane Studios','https://arkane-studios.com'),
('MachineGames','https://machinegames.com'),
('Remedy Entertainment','https://remedygames.com'),
('Creative Assembly','https://creative-assembly.com'),
('Obsidian Entertainment','https://obsidian.net'),
('Respawn Entertainment','https://respawn.com');

INSERT INTO Game VALUES
(70000,'2.0','download/cyberpunk','Cyberpunk 2077',59.99,5),
(80000,'1.0','download/gta5','Grand Theft Auto V',39.99,6),
(45000,'1.5','download/skyrim','The Elder Scrolls V: Skyrim',29.99,4),
(60000,'1.0','download/witcher3','The Witcher 3',39.99,5),
(30000,'1.0','download/portal2','Portal 2',19.99,3),
```

```
(55000,'1.0','download/doom','DOOM Eternal',39.99,4),
(50000,'1.0','download/eldenring','Elden Ring',59.99,43),
(48000,'1.0','download/spiderman','Spider-Man Remastered',49.99,41),
(35000,'1.0','download/baldursgate3','Baldur''s Gate 3',59.99,40),
(40000,'1.0','download/acvalhalla','Assassin''s Creed Valhalla',49.99,1),
(42000,'1.0','download/rdr2','Red Dead Redemption 2',59.99,6),
(30000,'1.0','download/minecraft','Minecraft',29.99,9),
(45000,'1.0','download/terraria','Terraria',14.99,38),
(47000,'1.0','download/hades','Hades',24.99,39),
(60000,'1.0','download/starfield','Starfield',69.99,4),
(33000,'1.0','download/overwatch','Overwatch 2',0,2),
(55000,'1.0','download/apex','Apex Legends',0,50),
(39000,'1.0','download/halo','Halo Infinite',59.99,17),
(41000,'1.0','download/destiny2','Destiny 2',0,17),
(50000,'1.0','download/fifa23','FIFA 23',59.99,2),
(52000,'1.0','download/nba2k','NBA 2K23',59.99,16),
(37000,'1.0','download/metroexodus','Metro Exodus',29.99,19),
(44000,'1.0','download/control','Control',29.99,47),
(41000,'1.0','download/deathstranding','Death Stranding',39.99,10),
(30000,'1.0','download/stardew','Stardew Valley',14.99,23),
(38000,'1.0','download/subnautica','Subnautica',29.99,20),
(43000,'1.0','download/fallout4','Fallout 4',29.99,4),
(46000,'1.0','download/dishonored','Dishonored 2',29.99,45),
(49000,'1.0','download/prey','Prey',29.99,45),
(47000,'1.0','download/wolfenstein','Wolfenstein II',39.99,46),
(48000,'1.0','download/battlefield1','Battlefield 1',39.99,2),
(52000,'1.0','download/battlefield5','Battlefield V',39.99,2),
(51000,'1.0','download/dragonage','Dragon Age Inquisition',29.99,2),
(55000,'1.0','download/mass','Mass Effect Legendary Edition',59.99,2),
(45000,'1.0','download/civ6','Civilization VI',29.99,16),
(46000,'1.0','download/xcom2','XCOM 2',29.99,16),
(47000,'1.0','download/totalwar','Total War Warhammer',59.99,48),
(30000,'1.0','download/hollowknight','Hollow Knight',14.99,24),
(32000,'1.0','download/celeste','Celeste',19.99,24),
(35000,'1.0','download/deadcells','Dead Cells',24.99,24),
(33000,'1.0','download/riskofrain','Risk of Rain 2',24.99,24),
(37000,'1.0','download/outerwilds','Outer Wilds',24.99,23),
(42000,'1.0','download/returnal','Returnal',59.99,10),
(39000,'1.0','download/godofwar','God of War',49.99,10),
(48000,'1.0','download/lastofus','The Last of Us Part I',59.99,42),
(47000,'1.0','download/uncharted','Uncharted Legacy of Thieves',49.99,42),
(41000,'1.0','download/sekiro','Sekiro Shadows Die Twice',59.99,43),
(43000,'1.0','download/darksouls3','Dark Souls III',39.99,43),
(45000,'1.0','download/bloodborne','Bloodborne',39.99,43),
(39000,'1.0','download/forza','Forza Horizon 5',59.99,17);
```

```sql
INSERT INTO Friend VALUES
('alex_k','maria_dev'),
('alex_k','john_gamer'),
('maria_dev','luna_play'),
('john_gamer','pixelking'),
('luna_play','shadowfox'),
('pixelking','nova_star'),
('shadowfox','armin_kh'),
('nova_star','tigergamer'),
('armin_kh','nightcoder'),
('tigergamer','stormblade'),
('nightcoder','silentwolf'),
('stormblade','cyberhero'),
('silentwolf','dragonlord'),
('cyberhero','retroplayer'),
('dragonlord','skygamer'),
('retroplayer','ironmaniac'),
('skygamer','darkrider'),
('ironmaniac','quantumplay'),
('darkrider','vortexgamer'),
('quantumplay','phantombyte'),
('vortexgamer','lunarfox'),
('phantombyte','silverarrow'),
('lunarfox','techwizard'),
('silverarrow','doomplayer'),
('techwizard','stealthninja'),
('doomplayer','orbitgamer'),
('stealthninja','hypernova'),
('orbitgamer','bytehunter'),
('hypernova','skyfall'),
('bytehunter','stormrider'),
('skyfall','pixelstorm'),
('stormrider','auroragamer'),
('pixelstorm','voidwalker'),
('auroragamer','frostgamer'),
('voidwalker','cyberninja'),
('frostgamer','neonplayer'),
('cyberninja','ultragamer'),
('neonplayer','gamemaster'),
('ultragamer','arcadehero'),
('gamemaster','binaryknight'),
('arcadehero','gamelord'),
('binaryknight','alphaomega'),
```

```
('gamelord','cosmicplayer'),
('alphaomega','vectorx'),
('cosmicplayer','speedrunner'),
('vectorx','battleking'),
('speedrunner','arcadefox'),
('battleking','nebulagamer'),
('arcadefox','legendplayer'),
('legendplayer','alex_k');


INSERT INTO Genre VALUES
('Action'),
('Adventure'),
('RPG'),
('Strategy'),
('Shooter'),
('Simulation'),
('Racing'),
('Sports'),
('Survival'),
('Horror'),
('Puzzle'),
('Platformer'),
('Stealth'),
('Open World'),
('Sandbox'),
('MMORPG'),
('Battle Royale'),
('Indie'),
('Sci-Fi'),
('Fantasy'),
('Historical'),
('Tactical'),
('Card Game'),
('Turn Based'),
('Real Time Strategy'),
('Roguelike'),
('Roguelite'),
('Metroidvania'),
('Arcade'),
('Casual'),
('Educational'),
('Party'),
('Co-op'),
('Multiplayer'),
```

```
('Singleplayer'),
('Narrative'),
('Survival Horror'),
('City Builder'),
('Space'),
('Military'),
('Fighting'),
('Rhythm'),
('Visual Novel'),
('Interactive Movie'),
('Deckbuilder'),
('Tower Defense'),
('Management'),
('Construction'),
('Physics'),
('Exploration');


INSERT INTO Game_Genre VALUES
(1,3),(2,14),(3,3),(4,3),(5,11),
(6,5),(7,3),(8,1),(9,3),(10,14),
(11,14),(12,15),(13,18),(14,26),(15,19),
(16,5),(17,17),(18,5),(19,5),(20,8),
(21,8),(22,5),(23,19),(24,36),(25,18),
(26,9),(27,3),(28,13),(29,19),(30,5),
(31,5),(32,5),(33,3),(34,3),(35,4),
(36,22),(37,25),(38,28),(39,18),(40,26),
(41,26),(42,50),(43,19),(44,1),(45,36),
(46,1),(47,1),(48,1),(49,1),(50,7);

INSERT INTO Developer VALUES
('CD Projekt Red','contact@cdprojekt.com'),
('Rockstar North','info@rockstargames.com'),
('Bethesda Game Studios','info@bethesda.net'),
('Valve','info@valvesoftware.com'),
('FromSoftware','info@fromsoftware.jp'),
('Insomniac Games','info@insomniac.games'),
('Larian Studios','info@larian.com'),
('Ubisoft Montreal','info@ubisoft.com'),
('Mojang Studios','info@mojang.com'),
('Re-Logic','info@relogic.com'),
('Supergiant Games','info@supergiantgames.com'),
('Respawn Entertainment','info@respawn.com'),
('343 Industries','info@halo.com'),
('Bungie','info@bungie.net'),
```

```
('EA Sports','info@ea.com'),
('Visual Concepts','info@2k.com'),
('4A Games','info@4agames.com'),
('Remedy Entertainment','info@remedy.com'),
('Kojima Productions','info@kojimaproductions.jp'),
('ConcernedApe','info@stardewvalley.net'),
('Unknown Worlds','info@unknownworlds.com'),
('Arkane Studios','info@arkane-studios.com'),
('MachineGames','info@machinegames.com'),
('DICE','info@dice.se'),
('BioWare','info@bioware.com'),
('Firaxis Games','info@firaxis.com'),
('Creative Assembly','info@creative-assembly.com'),
('Team Cherry','info@teamcherry.com.au'),
('Extremely OK Games','info@celestegame.com'),
('Motion Twin','info@motiontwin.com'),
('Hopoo Games','info@hopoogames.com'),
('Mobius Digital','info@mobiusdigitalgames.com'),
('Housemarque','info@housemarque.com'),
('Santa Monica Studio','info@sony.com'),
('Naughty Dog','info@naughtydog.com'),
('Playground Games','info@playground-games.com'),
('Obsidian Entertainment','info@obsidian.net'),
('id Software','info@idsoftware.com'),
('Guerrilla Games','info@guerrilla-games.com'),
('Sucker Punch','info@suckerpunch.com'),
('Infinity Ward','info@infinityward.com'),
('Treyarch','info@treyarch.com'),
('IO Interactive','info@ioi.dk'),
('Avalanche Studios','info@avalanchestudios.com'),
('Fatshark','info@fatshark.se'),
('Techland','info@techland.net'),
('Behaviour Interactive','info@bhvr.com'),
('Bluepoint Games','info@bluepointgames.com'),
('Asobo Studio','info@asobostudio.com'),
('People Can Fly','info@peoplecanfly.com');


INSERT INTO Game_Developer VALUES
(1,1),(2,2),(3,3),(4,1),(5,4),
(6,38),(7,5),(8,6),(9,7),(10,8),
(11,2),(12,9),(13,10),(14,11),(15,3),
(16,12),(17,12),(18,13),(19,14),(20,15),
(21,16),(22,17),(23,18),(24,19),(25,20),
(26,21),(27,3),(28,22),(29,22),(30,23),
```

```
(31,24),(32,24),(33,25),(34,25),(35,26),
(36,26),(37,27),(38,28),(39,29),(40,30),
(41,31),(42,32),(43,33),(44,34),(45,35),
(46,35),(47,5),(48,5),(49,5),(50,36);


INSERT INTO Library VALUES
('alex_k',1,120,1),
('maria_dev',2,240,0),
('john_gamer',3,85,1),
('luna_play',4,340,1),
('pixelking',5,60,0),
('shadowfox',6,200,1),
('nova_star',7,150,0),
('armin_kh',8,95,1),
('tigergamer',9,40,0),
('nightcoder',10,220,1),
('stormblade',11,300,1),
('silentwolf',12,120,0),
('cyberhero',13,180,1),
('dragonlord',14,90,0),
('retroplayer',15,200,1),
('skygamer',16,140,0),
('ironmaniac',17,160,1),
('darkrider',18,75,0),
('quantumplay',19,210,1),
('vortexgamer',20,180,0),
('phantombyte',21,60,1),
('lunarfox',22,50,0),
('silverarrow',23,100,1),
('techwizard',24,240,1),
('doomplayer',25,310,1),
('stealthninja',26,70,0),
('orbitgamer',27,190,1),
('hypernova',28,110,0),
('bytehunter',29,80,1),
('skyfall',30,120,0),
('stormrider',31,150,1),
('pixelstorm',32,200,0),
('auroragamer',33,90,1),
('voidwalker',34,300,1),
('frostgamer',35,130,0),
('cyberninja',36,140,1),
('neonplayer',37,60,0),
('ultragamer',38,220,1),
```

```
('gamemaster',39,95,0),
('arcadehero',40,180,1),
('binaryknight',41,210,0),
('gamelord',42,75,1),
('alphaomega',43,90,0),
('cosmicplayer',44,260,1),
('vectorx',45,170,0),
('speedrunner',46,80,1),
('battleking',47,150,0),
('arcadefox',48,100,1),
('nebulagamer',49,60,0),
('legendplayer',50,240,1);


INSERT INTO Wishlist VALUES
('alex_k',2),('maria_dev',3),('john_gamer',4),('luna_play',5),
('pixelking',6),('shadowfox',7),('nova_star',8),('armin_kh',9),
('tigergamer',10),('nightcoder',11),('stormblade',12),('silentwolf',13),
('cyberhero',14),('dragonlord',15),('retroplayer',16),('skygamer',17),
('ironmaniac',18),('darkrider',19),('quantumplay',20),('vortexgamer',21),
('phantombyte',22),('lunarfox',23),('silverarrow',24),('techwizard',25),
('doomplayer',26),('stealthninja',27),('orbitgamer',28),('hypernova',29),
('bytehunter',30),('skyfall',31),('stormrider',32),('pixelstorm',33),
('auroragamer',34),('voidwalker',35),('frostgamer',36),('cyberninja',37),
('neonplayer',38),('ultragamer',39),('gamemaster',40),('arcadehero',41),
('binaryknight',42),('gamelord',43),('alphaomega',44),('cosmicplayer',45),
('vectorx',46),('speedrunner',47),('battleking',48),('arcadefox',49),
('nebulagamer',50),('legendplayer',1);


INSERT INTO Review VALUES
(9,'Amazing open world experience','2024-01-10','alex_k',1),
(10,'One of the best games ever','2024-01-11','maria_dev',2),
(8,'Still fantastic after many years','2024-01-12','john_gamer',3),
(10,'Masterpiece storytelling','2024-01-13','luna_play',4),
(9,'Great puzzles and humor','2024-01-14','pixelking',5),
(8,'Fast paced shooter','2024-01-15','shadowfox',6),
(10,'Epic RPG combat','2024-01-16','nova_star',7),
(9,'Beautiful graphics','2024-01-17','armin_kh',8),
(10,'Huge RPG world','2024-01-18','tigergamer',9),
(8,'Fun open world gameplay','2024-01-19','nightcoder',10),
(10,'Incredible story','2024-01-20','stormblade',11),
(9,'Timeless sandbox','2024-01-21','silentwolf',12),
(9,'Great indie game','2024-01-22','cyberhero',13),
(10,'Addictive gameplay','2024-01-23','dragonlord',14),
```

```
(8,'Very ambitious','2024-01-24','retroplayer',15),
(7,'Competitive shooter','2024-01-25','skygamer',16),
(9,'Great battle royale','2024-01-26','ironmaniac',17),
(8,'Classic FPS gameplay','2024-01-27','darkrider',18),
(7,'Good multiplayer','2024-01-28','quantumplay',19),
(8,'Fun sports game','2024-01-29','vortexgamer',20),
(8,'Basketball simulation','2024-01-30','phantombyte',21),
(9,'Atmospheric shooter','2024-02-01','lunarfox',22),
(8,'Great narrative','2024-02-02','silverarrow',23),
(9,'Unique experience','2024-02-03','techwizard',24),
(10,'Relaxing farming game','2024-02-04','doomplayer',25),
(9,'Great underwater world','2024-02-05','stealthninja',26),
(8,'Classic RPG','2024-02-06','orbitgamer',27),
(9,'Excellent stealth mechanics','2024-02-07','hypernova',28),
(8,'Sci-fi immersive gameplay','2024-02-08','bytehunter',29),
(9,'Fantastic action shooter','2024-02-09','skyfall',30),
(7,'Large multiplayer battles','2024-02-10','stormrider',31),
(7,'Better with friends','2024-02-11','pixelstorm',32),
(8,'Great RPG story','2024-02-12','auroragamer',33),
(9,'Remastered masterpiece','2024-02-13','voidwalker',34),
(9,'Deep strategy gameplay','2024-02-14','frostgamer',35),
(8,'Challenging tactics','2024-02-15','cyberninja',36),
(9,'Epic fantasy battles','2024-02-16','neonplayer',37),
(10,'One of the best indie games','2024-02-17','ultragamer',38),
(9,'Very emotional story','2024-02-18','gamemaster',39),
(8,'Fast paced rogue gameplay','2024-02-19','arcadehero',40),
(9,'Great co-op action','2024-02-20','binaryknight',41),
(10,'Amazing exploration','2024-02-21','gamelord',42),
(8,'Beautiful sci-fi','2024-02-22','alphaomega',43),
(9,'Great combat','2024-02-23','cosmicplayer',44),
(10,'Emotional storytelling','2024-02-24','vectorx',45),
(9,'Fun adventure','2024-02-25','speedrunner',46),
(10,'Very challenging','2024-02-26','battleking',47),
(9,'Hard but rewarding','2024-02-27','arcadefox',48),
(8,'Classic action','2024-02-28','nebulagamer',49),
(9,'Great racing','2024-02-29','legendplayer',50);


INSERT INTO [Order] VALUES
(1,'2024-01-05','alex_k'),
(1,'2024-01-06','maria_dev'),
(2,'2024-01-07','john_gamer'),
(1,'2024-01-08','luna_play'),
(3,'2024-01-09','pixelking'),
(1,'2024-01-10','shadowfox'),
```

```
(1,'2024-01-11','nova_star'),
(2,'2024-01-12','armin_kh'),
(1,'2024-01-13','tigergamer'),
(1,'2024-01-14','nightcoder'),
(2,'2024-01-15','stormblade'),
(1,'2024-01-16','silentwolf'),
(1,'2024-01-17','cyberhero'),
(1,'2024-01-18','dragonlord'),
(2,'2024-01-19','retroplayer'),
(1,'2024-01-20','skygamer'),
(1,'2024-01-21','ironmaniac'),
(2,'2024-01-22','darkrider'),
(1,'2024-01-23','quantumplay'),
(1,'2024-01-24','vortexgamer'),
(2,'2024-01-25','phantombyte'),
(1,'2024-01-26','lunarfox'),
(1,'2024-01-27','silverarrow'),
(1,'2024-01-28','techwizard'),
(2,'2024-01-29','doomplayer'),
(1,'2024-01-30','stealthninja'),
(1,'2024-01-31','orbitgamer'),
(1,'2024-02-01','hypernova'),
(2,'2024-02-02','bytehunter'),
(1,'2024-02-03','skyfall'),
(1,'2024-02-04','stormrider'),
(2,'2024-02-05','pixelstorm'),
(1,'2024-02-06','auroragamer'),
(1,'2024-02-07','voidwalker'),
(1,'2024-02-08','frostgamer'),
(2,'2024-02-09','cyberninja'),
(1,'2024-02-10','neonplayer'),
(1,'2024-02-11','ultragamer'),
(2,'2024-02-12','gamemaster'),
(1,'2024-02-13','arcadehero'),
(1,'2024-02-14','binaryknight'),
(2,'2024-02-15','gamelord'),
(1,'2024-02-16','alphaomega'),
(1,'2024-02-17','cosmicplayer'),
(2,'2024-02-18','vectorx'),
(1,'2024-02-19','speedrunner'),
(1,'2024-02-20','battleking'),
(2,'2024-02-21','arcadefox'),
(1,'2024-02-22','nebulagamer'),
(1,'2024-02-23','legendplayer');
```

```sql
INSERT INTO Order_Game VALUES
(1,1,59.99),(2,2,39.99),(3,3,29.99),(4,4,39.99),(5,5,19.99),
(6,6,39.99),(7,7,59.99),(8,8,49.99),(9,9,59.99),(10,10,49.99),
(11,11,59.99),(12,12,29.99),(13,13,14.99),(14,14,24.99),(15,15,69.99),
(16,16,0),(17,17,0),(18,18,59.99),(19,19,0),(20,20,59.99),
(21,21,59.99),(22,22,29.99),(23,23,29.99),(24,24,39.99),(25,25,14.99),
(26,26,29.99),(27,27,29.99),(28,28,29.99),(29,29,29.99),(30,30,39.99),
(31,31,39.99),(32,32,39.99),(33,33,29.99),(34,34,59.99),(35,35,29.99),
(36,36,29.99),(37,37,59.99),(38,38,14.99),(39,39,19.99),(40,40,24.99),
(41,41,24.99),(42,42,24.99),(43,43,59.99),(44,44,49.99),(45,45,59.99),
(46,46,49.99),(47,47,59.99),(48,48,39.99),(49,49,39.99),(50,50,59.99);


INSERT INTO Payment VALUES
(1,59.99,'Credit Card'),
(2,39.99,'PayPal'),
(3,29.99,'Debit Card'),
(4,39.99,'Credit Card'),
(5,19.99,'PayPal'),
(6,39.99,'Debit Card'),
(7,59.99,'Credit Card'),
(8,49.99,'PayPal'),
(9,59.99,'Credit Card'),
(10,49.99,'Debit Card'),
(11,59.99,'Credit Card'),
(12,29.99,'PayPal'),
(13,14.99,'Credit Card'),
(14,24.99,'Debit Card'),
(15,69.99,'Credit Card'),
(16,0,'Free'),
(17,0,'Free'),
(18,59.99,'Credit Card'),
(19,0,'Free'),
(20,59.99,'Debit Card'),
(21,59.99,'Credit Card'),
(22,29.99,'PayPal'),
(23,29.99,'Credit Card'),
(24,39.99,'Debit Card'),
(25,14.99,'PayPal'),
(26,29.99,'Credit Card'),
(27,29.99,'Credit Card'),
(28,29.99,'Debit Card'),
(29,29.99,'Credit Card'),
(30,39.99,'PayPal'),
(31,39.99,'Credit Card'),
```

```
(32,39.99,'Debit Card'),
(33,29.99,'Credit Card'),
(34,59.99,'PayPal'),
(35,29.99,'Credit Card'),
(36,29.99,'Debit Card'),
(37,59.99,'Credit Card'),
(38,14.99,'PayPal'),
(39,19.99,'Credit Card'),
(40,24.99,'Debit Card'),
(41,24.99,'Credit Card'),
(42,24.99,'PayPal'),
(43,59.99,'Credit Card'),
(44,49.99,'Debit Card'),
(45,59.99,'Credit Card'),
(46,49.99,'PayPal'),
(47,59.99,'Credit Card'),
(48,39.99,'Debit Card'),
(49,39.99,'Credit Card'),
(50,59.99,'PayPal');
```

These DML statements populate the GamePlatformDB database with realistic sample data. They insert records into all tables while respecting relational constraints, enabling testing of queries and demonstrating system functionality.

3.4 Retrieve information using DQL queries & Implement Indexes to optimize query performance

1. Group By

```
-- Add index on userID in Library table (to optimize GROUP BY and JOIN)
CREATE INDEX idx_userID_library ON GamePlatformDB.dbo.Library(userID);

-- Query to get the total number of games owned by each user
SELECT u.username, COUNT(l.gameID) AS total_games
FROM GamePlatformDB.dbo.[User] u
JOIN GamePlatformDB.dbo.Library l ON u.username = l.userID
GROUP BY u.username;
```

	username	total_games
1	alex_k	1
2	alphaomega	1
3	arcadefox	2
4	arcadehero	1
5	armin_kh	1
6	auroragamer	1
7	battleking	1
8	binaryknight	1
9	bytehunter	1
10	cosmicplayer	1
11	cyberhero	1
12	cyberninja	1
13	darkrider	1

2. Group by Having

```
-- Add index on publisherID in Game table (to optimize GROUP BY and JOIN)
CREATE INDEX idx_publisherID_game ON GamePlatformDB.dbo.Game(publisherID);

-- Query to get the number of games each publisher has, but only for publishers
with more than 3 games
SELECT p.publisherName, COUNT(g.gameID) AS game_count
FROM GamePlatformDB.dbo.Publisher p
JOIN GamePlatformDB.dbo.Game g ON p.publisherID = g.publisherID
GROUP BY p.publisherName
HAVING COUNT(g.gameID) > 3;
```

	username	total_games
1	alex_k	1
2	alphaomega	1
3	arcadefox	2
4	arcadehero	1
5	armin_kh	1
6	auroragamer	1
7	battleking	1
8	binaryknight	1
9	bytehunter	1
10	cosmicplayer	1
11	cyberhero	1
12	cyberninja	1
13	darkrider	1

3. Join

```sql
-- Add index on publisherID in Game table (to optimize JOIN)
CREATE INDEX idx_publisherID ON GamePlatformDB.dbo.Game(publisherID);

-- Query to retrieve all games along with their publisher names
SELECT g.gameName, p.publisherName
FROM GamePlatformDB.dbo.Game g
JOIN GamePlatformDB.dbo.Publisher p ON g.publisherID = p.publisherID;
```

	gameName	publisherName
4	The Witcher 3	CD Projekt
5	Portal 2	Valve
6	DOOM Eternal	Bethesda Softworks
7	Elden Ring	FromSoftware
8	Spider-Man Remastered	Insomniac Games
9	Baldur's Gate 3	Larian Studios
10	Assassin's Creed Valhalla	Ubisoft
11	Red Dead Redemption 2	Rockstar Games
12	Minecraft	Nintendo
13	Terraria	Re-Logic
14	Hades	Supergiant Games
15	Starfield	Bethesda Softworks
16	Overwatch 2	Electronic Arts
17	Apex Legends	Respawn Entertainment
18	Halo Infinite	Warner Bros Games
19	Destiny 2	Warner Bros Games
20	FIFA 23	Electronic Arts
21	NBA 2K23	2K Games
22	Metro Exodus	Deep Silver
23	Control	Remedy Entertainment
24	Death Stranding	Sony Interactive Entert...
25	Stardew Valley	Annapurna Interactive
26	Subnautica	505 Games
27	Fallout 4	Bethesda Softworks
28	Dishonored 2	Arkane Studios
29	Prey	Arkane Studios
30	Wolfenstein II	MachineGames

4. Joining and group by having

```
-- Add index on orderID in Order_Game table (to optimize JOIN)
CREATE INDEX idx_orderID_order_game ON GamePlatformDB.dbo.Order_Game(orderID);

-- Add index on userID in Order table (to optimize JOIN)
CREATE INDEX idx_userID_order ON GamePlatformDB.dbo.[Order](userID);

-- Query to retrieve the total amount spent by each user on games
SELECT o.userID, SUM(og.priceAtPurchase) AS total_spent
FROM GamePlatformDB.dbo.[Order] o
JOIN GamePlatformDB.dbo.Order_Game og ON o.orderID = og.orderID
GROUP BY o.userID
HAVING SUM(og.priceAtPurchase) > 40;
```

	userID	total_spent
1	alex_k	59.99
2	alphaomega	59.99
3	armin_kh	49.99
4	battleking	59.99
5	cosmicplayer	49.99
6	darkrider	59.99
7	legendplayer	59.99
8	neonplayer	59.99
9	nightcoder	49.99
10	nova_star	59.99
11	phantombyte	59.99
12	retroplayer	69.99
13	speedrunner	49.99
14	stormblade	59.99
15	tigergamer	59.99
16	vectorx	59.99
17	voidwalker	59.99
18	vortexgamer	59.99

5. Using Join with String aggregate

```
-- Add index on gameID in Game_Genre table (to optimize JOIN)
CREATE INDEX idx_gameID_genre ON GamePlatformDB.dbo.Game_Genre(gameID);

-- Add index on genreName in Genre table (to optimize WHERE filtering)
CREATE INDEX idx_genreName ON GamePlatformDB.dbo.Genre(genreName);

-- Add index on developerID in Game_Developer table (to optimize JOIN)
CREATE INDEX idx_developerID_game_developer ON
GamePlatformDB.dbo.Game_Developer(developerID);

-- Add index on gameID in Game_Developer table (to optimize JOIN)
CREATE INDEX idx_gameID_game_developer ON
GamePlatformDB.dbo.Game_Developer(gameID);
```

```sql
-- Query to get the names of games, their genres, and developers for all "RPG"
games
SELECT g.gameName, ge.genreName, STRING_AGG(d.developerName, ', ') AS developers
FROM GamePlatformDB.dbo.Game g
JOIN GamePlatformDB.dbo.Game_Genre gg ON g.gameID = gg.gameID
JOIN GamePlatformDB.dbo.Genre ge ON gg.genreID = ge.genreID
JOIN GamePlatformDB.dbo.Game_Developer gd ON g.gameID = gd.gameID
JOIN GamePlatformDB.dbo.Developer d ON gd.developerID = d.developerID
WHERE ge.genreName = 'RPG'
GROUP BY g.gameName, ge.genreName;
```

	gameName	genreName	developers
1	Baldur's Gate 3	RPG	Respawn Entertainment
2	Cyberpunk 2077	RPG	CD Projekt Red, Valve
3	Dragon Age Inquisition	RPG	IO Interactive
4	Elden Ring	RPG	Mojang Studios
5	Fallout 4	RPG	Obsidian Entertainment
6	Mass Effect Legendary Edition	RPG	Avalanche Studios
7	The Elder Scrolls V: Skyrim	RPG	Bethesda Game Studios, EA Sports, Creative Asse...
8	The Witcher 3	RPG	FromSoftware

6. Using Join with Where

```sql
-- Add index on gameName in Game table (to optimize WHERE filtering)
CREATE INDEX idx_gameName ON GamePlatformDB.dbo.Game(gameName);

-- Index idx_userID_library already created above (reused here to optimize JOIN)
CREATE INDEX -- Index idx_userID_library already exists (created in query 1) ON
GamePlatformDB.dbo.Library(userID);

-- Query to retrieve usernames of users who own "Cyberpunk 2077"
SELECT u.username
FROM GamePlatformDB.dbo.[User] u
JOIN GamePlatformDB.dbo.Library l ON u.username = l.userID
JOIN GamePlatformDB.dbo.Game g ON l.gameID = g.gameID
WHERE g.gameName = 'Cyberpunk 2077';
```

	username
1	alex_k

7. Ordering

```sql
-- Add index on gamePrice in Game table (to optimize ORDER BY)
CREATE INDEX idx_gamePrice ON GamePlatformDB.dbo.Game(gamePrice);

-- Query to retrieve all games sorted by price, from highest to lowest
SELECT gameName, gamePrice
FROM GamePlatformDB.dbo.Game
ORDER BY gamePrice DESC;
```

	gameName	gamePrice
4	Cyberpunk 2077	59.99
5	Elden Ring	59.99
6	Baldur's Gate 3	59.99
7	FIFA 23	59.99
8	NBA 2K23	59.99
9	Mass Effect Legendary Edition	59.99
10	Total War Warhammer	59.99
11	Returnal	59.99
12	The Last of Us Part I	59.99
13	Sekiro Shadows Die Twice	59.99
14	Forza Horizon 5	59.99
15	Uncharted Legacy of Thieves	49.99
16	God of War	49.99
17	Assassin's Creed Valhalla	49.99
18	Spider-Man Remastered	49.99
19	Grand Theft Auto V	39.99
20	The Witcher 3	39.99
21	DOOM Eternal	39.99
22	Death Stranding	39.99
23	Wolfenstein II	39.99
24	Battlefield 1	39.99
25	Battlefield V	39.99
26	Dark Souls III	39.99
27	Bloodborne	39.99

8. Ordering By Top 5

```sql
-- Index idx_gamePrice already created above (reused here to optimize ORDER BY)
CREATE INDEX -- Index idx_gamePrice already exists (created in query 7) ON
GamePlatformDB.dbo.Game(gamePrice);

-- Query to retrieve the top 5 most expensive games
SELECT TOP 5 gameName, gamePrice
FROM GamePlatformDB.dbo.Game
```

```
ORDER BY gamePrice DESC;
```

	gameName	gamePrice
1	Starfield	69.99
2	Cyberpunk 2077	59.99
3	Elden Ring	59.99
4	Baldur's Gate 3	59.99
5	Red Dead Redemption 2	59.99

9. Using Select

```
--Select all usernames and emails from User
SELECT username, email, balance
FROM GamePlatformDB.dbo.[User];
```

	username	email	balance
1	alex_k	alexk@gmail.com	120.50
2	alphaomega	alpha@gmail.com	160.60
3	arcadefox	arcfox@gmail.com	110.00
4	arcadehero	arcade@gmail.com	140.40
5	armin_kh	armin.kh@gmail.com	50.00
6	auroragamer	aurora@gmail.com	125.75
7	battleking	battle@gmail.com	260.75
8	binaryknight	binary@gmail.com	190.90
9	bytehunter	hunter@gmail.com	55.55
10	cosmicplayer	cosmic@gmail.com	220.20
11	cyberhero	cyberhero@gmail.com	500.00
12	cyberninja	cyberninja@gmail.com	210.00
13	darkrider	dark@gmail.com	110.10
14	doomplayer	doom@gmail.com	70.00
15	dragonlord	dragon@gmail.com	275.45
16	frostgamer	frost@gmail.com	60.00
17	gamelord	lord@gmail.com	275.00
18	gamemaster	gm@gmail.com	500.50
19	hypernova	hyper@gmail.com	310.10
20	ironmaniac	iron@gmail.com	320.80
21	john_gamer	john.gamer@gmail.com	75.25
22	legendplayer	legend@gmail.com	350.00
23	luna_play	luna.play@gmail.com	180.00
24	lunarfox	lunar@gmail.com	88.88

10. Using Where

```
-- Index idx_gamePrice already created above (reused here to optimize WHERE
filtering)
CREATE INDEX -- Index idx_gamePrice already exists (created in query 7) ON
GamePlatformDB.dbo.Game(gamePrice);

-- Query to retrieve all games that cost more than $50
SELECT gameName, gamePrice
FROM GamePlatformDB.dbo.Game
WHERE gamePrice > 50;
```

	gameName	gamePrice
1	Cyberpunk 2077	59.99
2	Elden Ring	59.99
3	Baldur's Gate 3	59.99
4	Red Dead Redemption 2	59.99
5	Starfield	69.99
6	Halo Infinite	59.99
7	FIFA 23	59.99
8	NBA 2K23	59.99
9	Mass Effect Legendary Edition	59.99
10	Total War Warhammer	59.99
11	Returnal	59.99
12	The Last of Us Part I	59.99
13	Sekiro Shadows Die Twice	59.99
14	Forza Horizon 5	59.99

3.5 Apply SQL queries and store results in Views

```
-- Displays a simplified list of all users by showing only their username and
email from the User table.
CREATE VIEW AllUsers AS
SELECT username, email
FROM [User];
```

```sql
-- Shows all games that cost more than 50, displaying their name and price
from the Game table.
CREATE VIEW ExpensiveGames AS
SELECT gameName, gamePrice
FROM Game
WHERE gamePrice > 50;
```

```sql
-- Displays each game along with its publisher name by joining the Game and
Publisher tables.
CREATE VIEW GameWithPublisher AS
SELECT g.gameName, p.publisherName
FROM Game g
JOIN Publisher p ON g.publisherID = p.publisherID;
```

```sql
-- Lists all RPG games, showing the game name, genre, and a combined list of
their developers.
CREATE VIEW RPG_Games AS
SELECT g.gameName, ge.genreName, STRING_AGG(d.developerName, ', ') AS
developers
FROM Game g
JOIN Game_Genre gg ON g.gameID = gg.gameID
JOIN Genre ge ON gg.genreID = ge.genreID
JOIN Game_Developer gd ON g.gameID = gd.gameID
JOIN Developer d ON gd.developerID = d.developerID
WHERE ge.genreName = 'RPG'
GROUP BY g.gameName, ge.genreName;
```

```sql
-- Calculates the total amount spent by each user by summing the prices of
all games they purchased.
CREATE VIEW TotalSpentByUsers AS
SELECT o.userID, SUM(og.priceAtPurchase) AS total_spent
FROM [Order] o
JOIN Order_Game og ON o.orderID = og.orderID
GROUP BY o.userID;
```

```sql
-- Lists the usernames of users who own “Cyberpunk 2077” in their game
library.
CREATE VIEW UsersOwnCyberpunk AS
SELECT u.username
FROM [User] u
```

```
JOIN Library l ON u.username = l.userID
JOIN Game g ON l.gameID = g.gameID
WHERE g.gameName = 'Cyberpunk 2077';
```

3.6 Implement Triggers to automate database operations and enforce business rules

1.

```
-- Trigger: Automatically add the game to the user's library after an order
is placed
CREATE TRIGGER add_game_to_library
ON Order_Game
AFTER INSERT
AS
BEGIN
    -- Insert the game into the Library table if it's not already owned
    INSERT INTO Library (userID, gameID, playTime, favorite)
    SELECT o.userID, og.gameID, 0, 0
    FROM [Order] o
    JOIN Order_Game og ON o.orderID = og.orderID
    WHERE og.orderID IN (SELECT orderID FROM inserted)
    AND NOT EXISTS (
        SELECT 1
        FROM Library l
        WHERE l.userID = o.userID AND l.gameID = og.gameID
    );
END;
```

2.

```
 -- Trigger: Prevent deletion of a game if it is in any user's library

CREATE TRIGGER prevent_game_deletion
ON Game
INSTEAD OF DELETE
AS
BEGIN
    DECLARE @gameID INT;
    SELECT @gameID = gameID FROM deleted;

    -- Check if the game is in the Library table
    IF EXISTS (SELECT 1 FROM Library WHERE gameID = @gameID)
    BEGIN
        -- If the game is owned by users, raise an error and prevent deletion
```

```
        RAISERROR('Cannot delete the game as it is owned by users.', 16, 1);
    END
    ELSE
    BEGIN
        -- If the game is not owned by any user, proceed with deletion
        DELETE FROM Game WHERE gameID = @gameID;
    END;
END;
```

3.

```
-- Trigger: Prevent inserting a review for a game that the user doesn't own
CREATE TRIGGER prevent_review_for_unowned_game
ON Review
INSTEAD OF INSERT
AS
BEGIN
    DECLARE @userID VARCHAR(50), @gameID INT;

    -- Get the values from the inserted review row
    SELECT @userID = userID, @gameID = gameID FROM inserted;

    -- Check if the user owns the game
    IF NOT EXISTS (SELECT 1 FROM Library WHERE userID = @userID AND gameID =
@gameID)
    BEGIN
        -- If the user doesn't own the game, raise an error
        RAISERROR('You can only review games you own.', 16, 1);
    END
    ELSE
    BEGIN
        -- Proceed with inserting the review if the user owns the game
        INSERT INTO Review (reviewID, rating, description, reviewDate,
userID, gameID)
        SELECT reviewID, rating, description, reviewDate, userID, gameID
        FROM inserted;
    END;
END;
```

4.

```
-- Trigger: Automatically update the playtime after a game is played
CREATE TRIGGER update_playtime
ON Library
```

```
AFTER UPDATE
AS
BEGIN
    -- Check if the playtime has increased
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN deleted d ON i.userID = d.userID AND i.gameID = d.gameID
        WHERE i.playTime > d.playTime
    )
    BEGIN
        -- Update the playtime for the game
        UPDATE Library
        SET playTime = (SELECT playTime FROM inserted WHERE userID =
Library.userID AND gameID = Library.gameID)
        WHERE EXISTS (
            SELECT 1
            FROM inserted
            WHERE inserted.userID = Library.userID AND inserted.gameID =
Library.gameID
        );
    END;
END;
```

5.

```
-- Trigger: Deduct user balance on purchase and block the transaction if
funds are insufficient.
CREATE TRIGGER trg_CheckUserBalance
ON GamePlatformDB.dbo.Order_Game
INSTEAD OF INSERT
AS
BEGIN
    -- Check if any inserted purchase exceeds user's balance
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN GamePlatformDB.dbo.[Order] o ON i.orderID = o.orderID
        JOIN GamePlatformDB.dbo.[User] u ON o.userID = u.username
        JOIN GamePlatformDB.dbo.Game g ON i.gameID = g.gameID
        WHERE u.balance < g.gamePrice * o.quantity
    )
    BEGIN
        -- Stop transaction if balance is insufficient
```

```
        RAISERROR ('Not enough balance to complete the purchase.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Insert purchase if balance is sufficient
    INSERT INTO GamePlatformDB.dbo.Order_Game (gameID, orderID,
priceAtPurchase)
    SELECT gameID, orderID, priceAtPurchase
    FROM inserted;

    -- Deduct total cost from user's balance
    UPDATE u
    SET u.balance = u.balance - (g.gamePrice * o.quantity)
    FROM GamePlatformDB.dbo.[User] u
    JOIN GamePlatformDB.dbo.[Order] o ON u.username = o.userID
    JOIN inserted i ON o.orderID = i.orderID
    JOIN GamePlatformDB.dbo.Game g ON i.gameID = g.gameID;
END;
```

3.7 Develop Stored Procedures for reusable database logic and operations

```
-- Stored Procedure: Add a game to the user's library
CREATE PROCEDURE AddGameToLibrary
    @userID VARCHAR(50),
    @gameID INT
AS
BEGIN
    -- Check if the user already owns the game
    IF EXISTS (SELECT 1 FROM Library WHERE userID = @userID AND gameID =
@gameID)
    BEGIN
        PRINT 'User already owns this game.';
        RETURN;
    END

    -- Insert the game into the library with default values for playtime and
favorite
    INSERT INTO Library (userID, gameID, playTime, favorite)
    VALUES (@userID, @gameID, 0, 0);

    PRINT 'Game successfully added to the library.';
END;
```

```sql
CREATE PROCEDURE AddNewGame
    @gameName VARCHAR(100),
    @gameSize INT,
    @version VARCHAR(20),
    @downloadURL VARCHAR(200),
    @gamePrice DECIMAL(10,2),
    @publisherID INT,
    @developerID INT -- DeveloperID passed as a parameter
AS
BEGIN
    -- Insert the new game into the Game table (gameID will auto-increment)
    INSERT INTO Game (gameName, gameSize, version, downloadURL, gamePrice,
publisherID)
    VALUES (@gameName, @gameSize, @version, @downloadURL, @gamePrice,
@publisherID);

    -- Retrieve the gameID of the last inserted game using SCOPE_IDENTITY
    DECLARE @gameID INT;
    SET @gameID = SCOPE_IDENTITY();  -- SCOPE_IDENTITY() gets the last
inserted identity value

    -- Check if gameID was successfully retrieved
    IF @gameID IS NULL
    BEGIN
        PRINT 'Failed to retrieve gameID.';
        RETURN; -- Exit the procedure if no gameID is returned
    END

    -- Insert into Game_Developer table using the gameID and developerID
    INSERT INTO Game_Developer (gameID, developerID)
    VALUES (@gameID, @developerID);

    PRINT 'Game successfully added with gameID ' + CAST(@gameID AS
VARCHAR(10));
END;
```

```sql
-- Stored Procedure: Get all games owned by a user
CREATE PROCEDURE GetGamesOwnedByUser
    @userID VARCHAR(50)
AS
BEGIN
    -- Retrieve all games owned by the user
    SELECT g.gameName, g.gamePrice, l.playTime
    FROM Game g
```

```
    JOIN Library l ON g.gameID = l.gameID
    WHERE l.userID = @userID;
END;
```

```
-- Stored Procedure: Get total amount spent by a user on games
CREATE PROCEDURE GetUserTotalSpend
    @userID VARCHAR(50)
AS
BEGIN
    -- Calculate the total amount spent by the user on all their orders
    SELECT SUM(og.priceAtPurchase) AS TotalAmountSpent
    FROM [Order] o
    JOIN Order_Game og ON o.orderID = og.orderID
    WHERE o.userID = @userID;
END;
```

```
-- Stored Procedure: Remove a game from the user's library
CREATE PROCEDURE RemoveGameFromLibrary
    @userID VARCHAR(50),
    @gameID INT
AS
BEGIN
    -- Check if the user owns the game
    IF NOT EXISTS (SELECT 1 FROM Library WHERE userID = @userID AND gameID =
@gameID)
    BEGIN
        PRINT 'User does not own this game.';
        RETURN;
    END

    -- Remove the game from the library
    DELETE FROM Library WHERE userID = @userID AND gameID = @gameID;

    PRINT 'Game successfully removed from the library.';
END;
```

```
-- Stored Procedure: Update the price of a game
CREATE PROCEDURE UpdateGamePrice
    @gameID INT,
    @newPrice DECIMAL(10, 2)
AS
BEGIN
    -- Update the price of the game
```

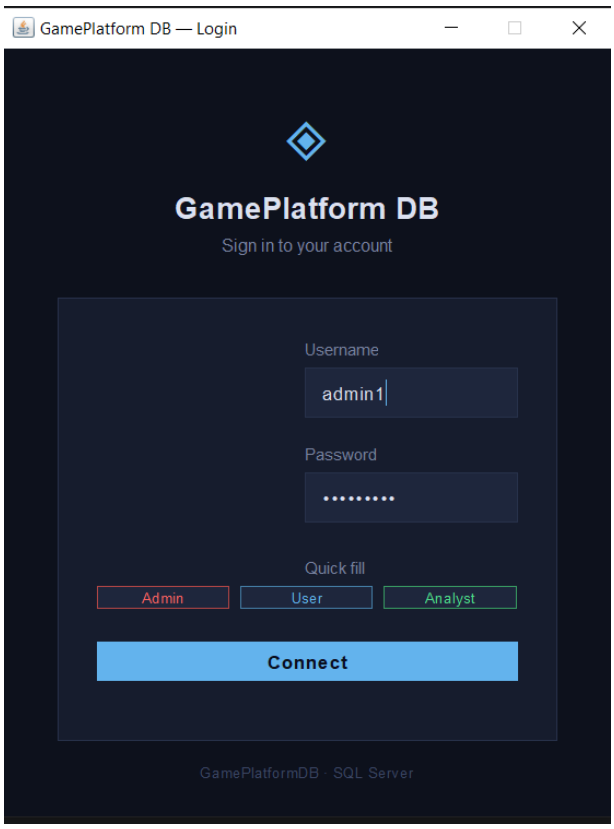
```
    UPDATE Game
    SET gamePrice = @newPrice
    WHERE gameID = @gameID;

    PRINT 'Game price updated successfully.';
END;
```

3.8 Implement access control using DCL statements

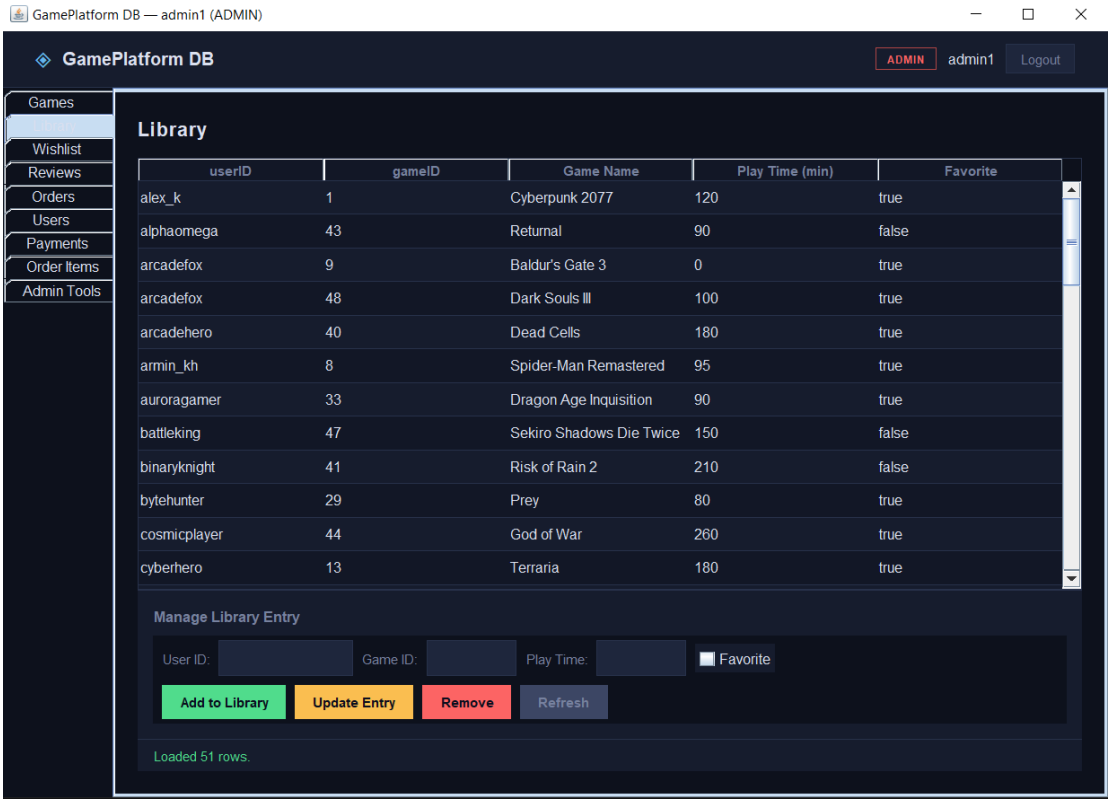
```
-- Create logins
CREATE LOGIN admin1 WITH PASSWORD = 'Admin123!';
CREATE LOGIN user1 WITH PASSWORD = 'User123!';
CREATE LOGIN analyst1 WITH PASSWORD = 'Analyst123!';


USE GamePlatformDB;
-- create users
CREATE USER admin1 FOR LOGIN admin1;
CREATE USER user1 FOR LOGIN user1;
CREATE USER analyst1 FOR LOGIN analyst1;

-- create roles
CREATE ROLE UserRole;
CREATE ROLE AnalystRole;

-- grant permissions

-- User
GRANT SELECT ON Game TO UserRole;
GRANT SELECT ON Genre TO UserRole;
GRANT SELECT ON Library TO UserRole;
GRANT SELECT ON Wishlist TO UserRole;
GRANT SELECT ON Review TO UserRole;
GRANT SELECT ON [Order] TO UserRole;
GRANT SELECT ON Publisher TO UserRole;
GRANT SELECT ON Developer TO UserRole;

GRANT INSERT ON Library TO UserRole;
GRANT INSERT ON Wishlist TO UserRole;
GRANT INSERT ON Review TO UserRole;
GRANT INSERT ON [Order] TO UserRole;

GRANT UPDATE ON Library TO UserRole;
GRANT UPDATE ON Review TO UserRole;
```

```sql
GRANT DELETE ON Wishlist TO UserRole;
GRANT DELETE ON Library TO UserRole;

-- Analyst
GRANT SELECT ON Game TO AnalystRole;
GRANT SELECT ON Genre TO AnalystRole;
GRANT SELECT ON [User] TO AnalystRole;
GRANT SELECT ON Library TO AnalystRole;
GRANT SELECT ON Wishlist TO AnalystRole;
GRANT SELECT ON Review TO AnalystRole;
GRANT SELECT ON [Order] TO AnalystRole;
GRANT SELECT ON Order_Game TO AnalystRole;
GRANT SELECT ON Payment TO AnalystRole;
GRANT SELECT ON Publisher TO AnalystRole;
GRANT SELECT ON Developer TO AnalystRole;

-- add users to roles


-- Admin (full control)
ALTER ROLE db_owner ADD MEMBER admin1;

ALTER ROLE UserRole ADD MEMBER user1;
ALTER ROLE AnalystRole ADD MEMBER analyst1;
```

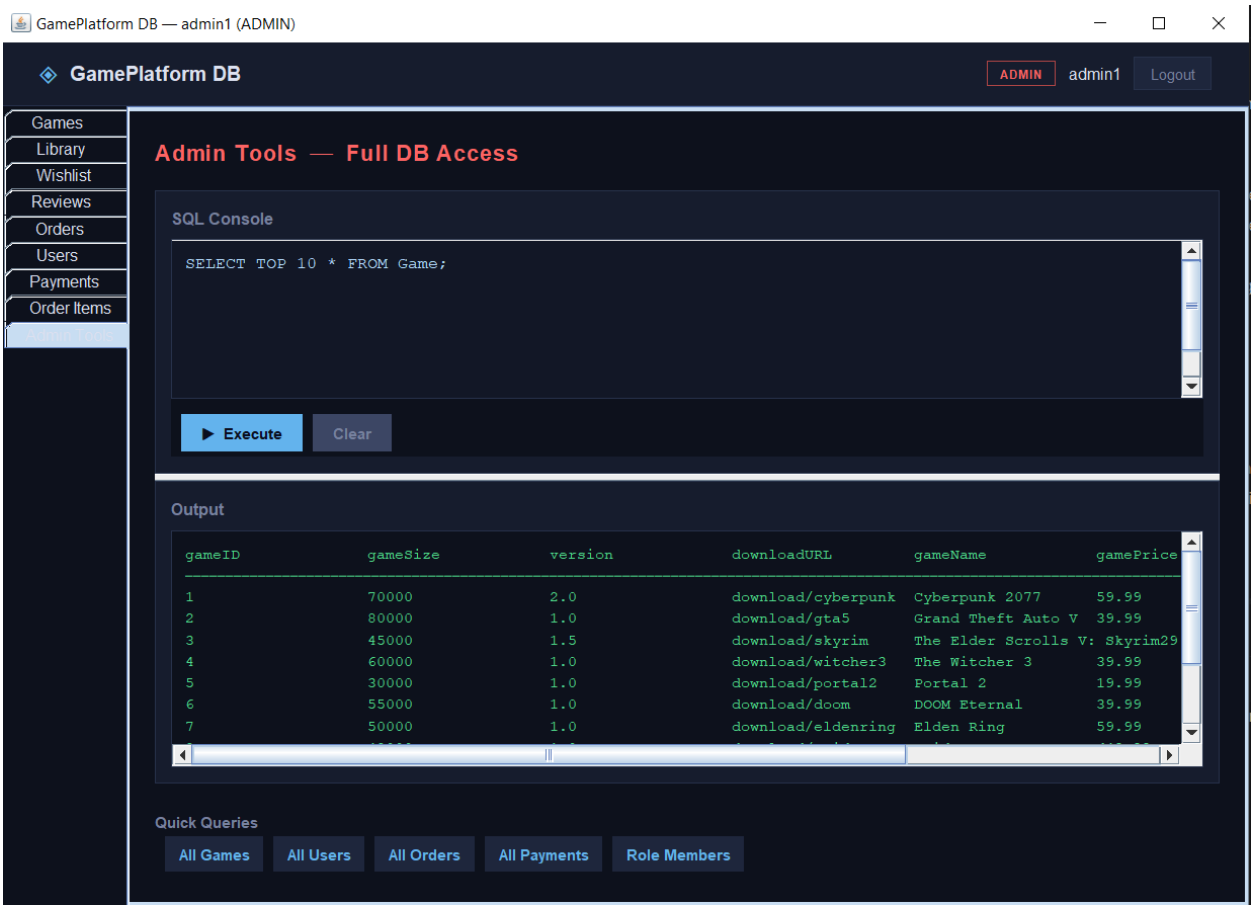
3.9 Connect the database to an application or interface



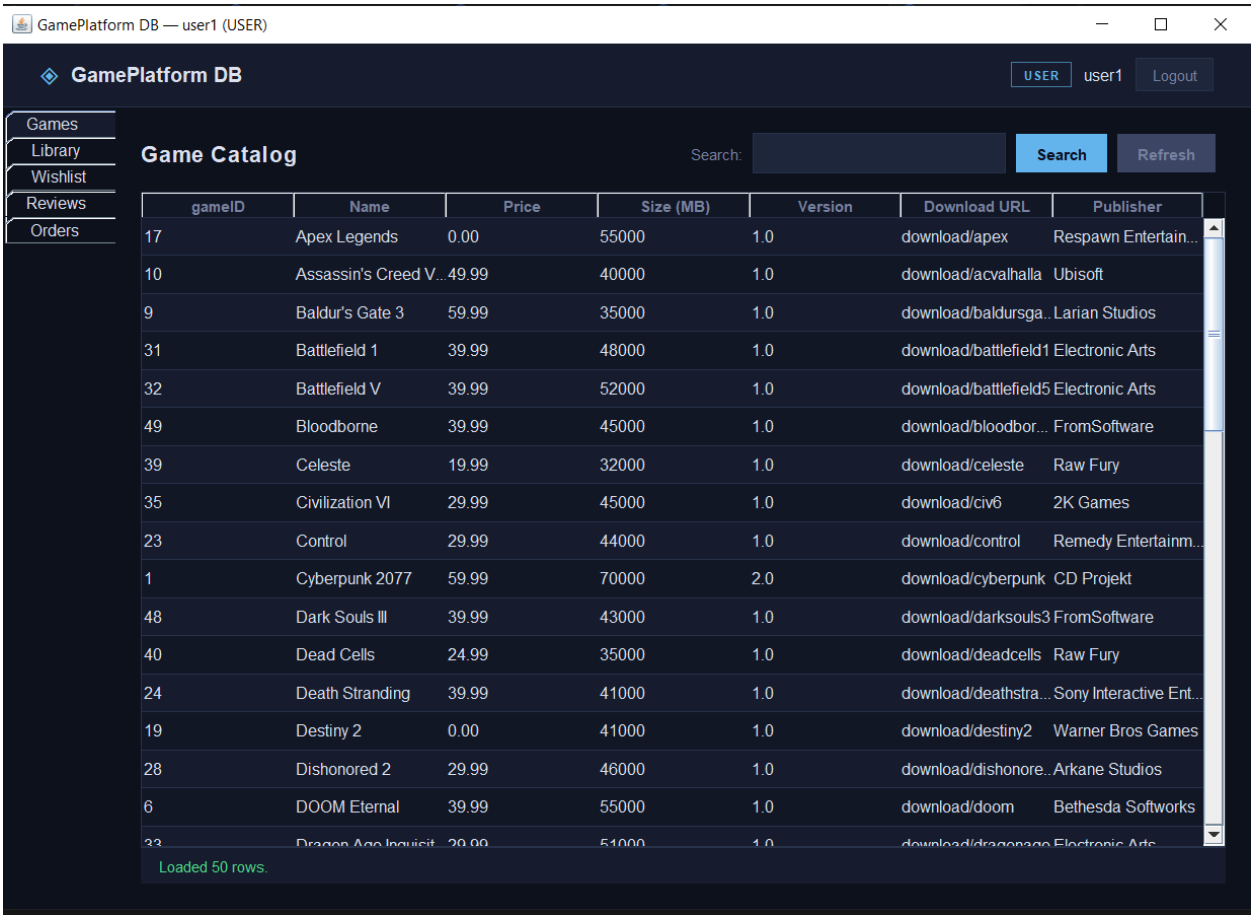
This is the login screen for database administrators.



The Library menu allows administrators to add, update, and remove games from users' libraries.

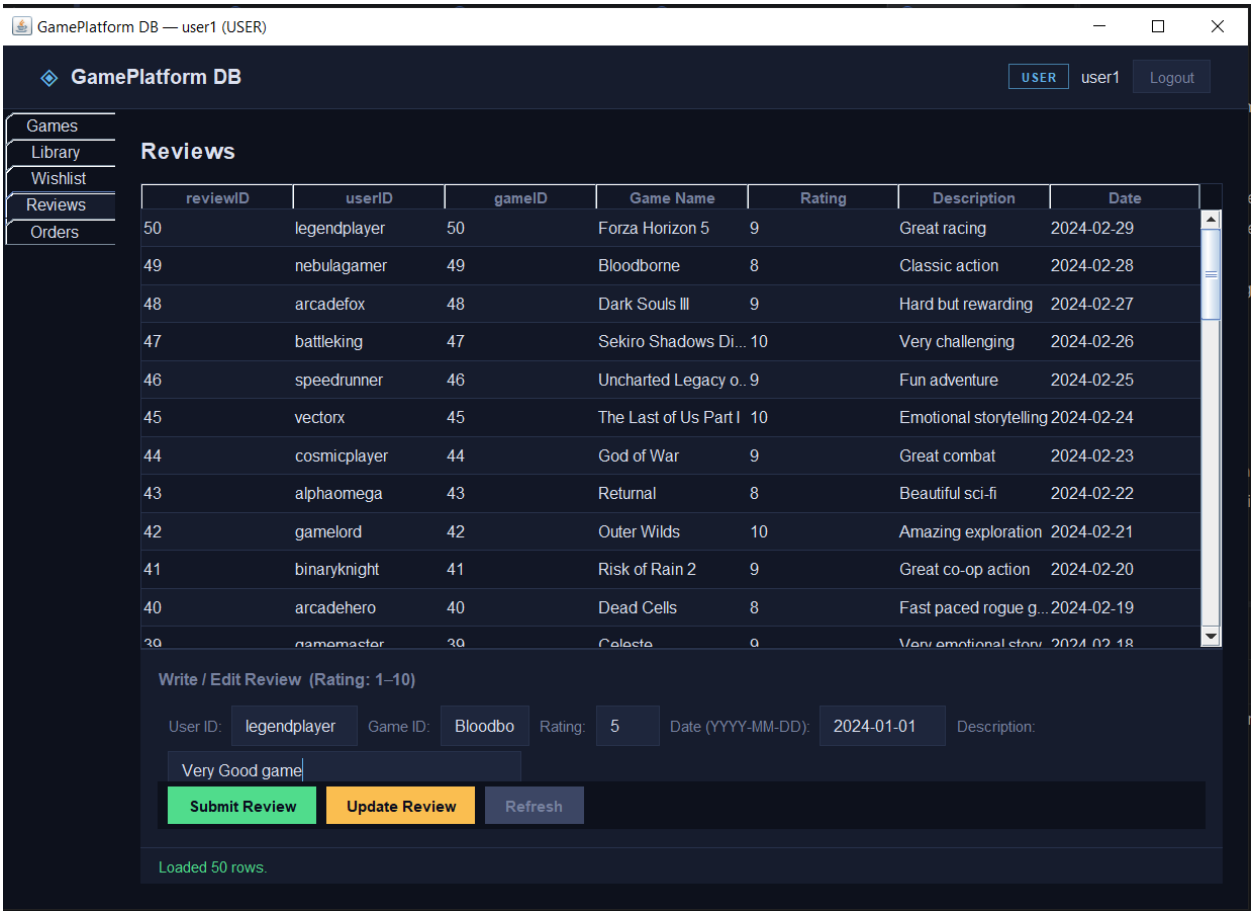


In the Admin Tools window, the administrator can write SQL queries manually to control the database.



Users can browse and search for games in the game catalog.

The Analyst view allows data analysis across different database tables.



Users can compose and update their game reviews from this view.

4. Conclusion

GamePlatformDB successfully implements a fully normalized relational database for a digital game distribution platform. The project covers the complete database lifecycle from conceptual design through physical implementation and application integration.

Key Achievements

- 14 tables designed in 3NF with no redundancy or transitive dependencies
- 50 records per table (650 total rows) providing realistic test data
- 6 views encapsulating common reporting and data access patterns
- 11 indexes targeting high-frequency JOIN, WHERE, and ORDER BY columns
- 5 triggers enforcing critical business rules automatically at the database level
- 6 stored procedures providing reusable, parameterized database operations
- Role-based access control with 3 principals (admin, user, analyst) and least-privilege grants
- 26 relational algebra queries demonstrating selection, projection, join, union, intersection, and difference
- Java Swing GUI application connecting via JDBC with role-aware interface

Design Decisions

Several deliberate design choices merit mention:

- Username as natural PK for User — avoids surrogate key, enforces uniqueness at the application level, and makes joins to Library, Wishlist, Review, Friend, and Order readable without additional lookups.
- Composite PKs for associative tables (Library, Wishlist, Game_Genre, Game_Developer, Friend, Order_Game) — avoids unnecessary surrogate IDs and ensures each combination is unique at the database level.
- priceAtPurchase in Order_Game — decouples historical purchase price from the current gamePrice, correctly modeling the fact that game prices change over time.
- INSTEAD OF triggers for deletion, review insertion, and balance validation — provide fine-grained control over DML operations, allowing custom error messages and conditional logic before the operation proceeds.

Limitations & Future Work

- The Friend table is directional — (alex, bob) and (bob, alex) are treated as different records. A bidirectional friendship model would require additional logic.
- Full-text search on game descriptions and reviews is not implemented; SQL Server Full-Text Search could be added.

- The application GUI does not yet support the Friend or Game_Developer management features.